

Advanced Data Management, VBScript Automation, and Relational Database Integration in WinCC SCADA Tutorial

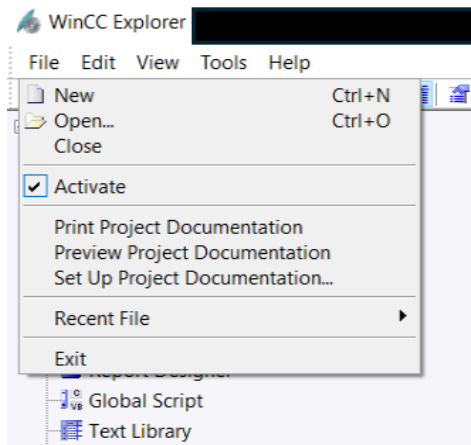
Prepared By
İclal Akman

In compliance with corporate cyber security policies, sensitive internal information—such as user directory IDs and local workstation pathnames—has been masked in the screenshots. This does not alter the technical functionality or data accuracy of the project.

Contents

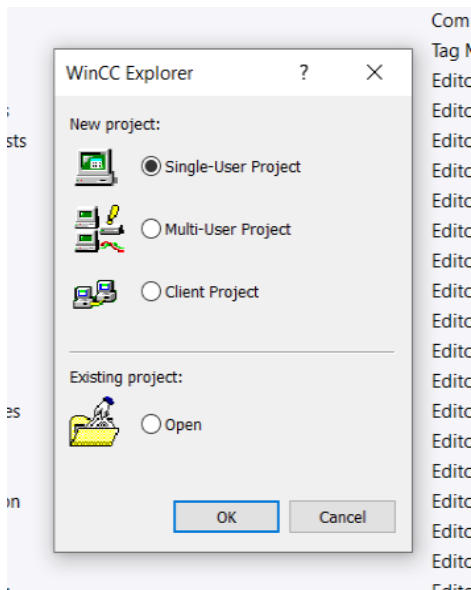
Creating a Project	3
Tag Management	4
Tag Logging and Data Archiving	6
Tag Logging - Simulation	7
Database Management with SQL Server Management Studio (SSMS)	14
Global Script	18
.....	18
Configurations	19
.....	20
External Database Filtering (iclal-db)	21
Investigating Internal WinCC Archives and Implementing Automated Tag Discovery	24
Exporting data as .txt form	27
Exporting data as .xls form	29
.....	29
Analyzing Multi-Cycle Archiving	31
Quality codes of Tags	34
.....	35
.....	35
Database Performance Optimization / Back-up	37
Role-Based User Management	45
.....	45
.....	45
HMI Navigation Design via Direct Connection	48
Runtime Security Validation and Dynamic User Login Testing	49

Creating a Project

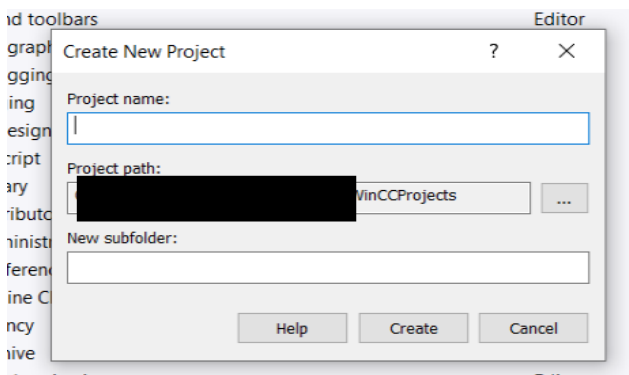


To start the SCADA system design, I used Siemens **SIMATIC WinCC Explorer**. This initial phase focused on setting up a new project foundation, configuring the environment, and managing the core files.

First, I opened WinCC Explorer and navigated it to the top menu. I clicked on **File** and selected **New** (as shown in **Figure 1**). This opened the setup guide for creating a fresh application

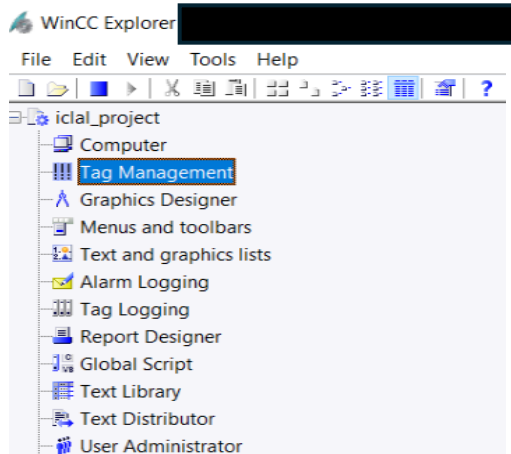


Next, a dialog box prompted me to select the project structure (**Figure 2**). For this specific application, I chose **Single-User Project**. This configuration ensures that the configuration and runtime operate locally on a single workstation, rather than a distributed client-server network.

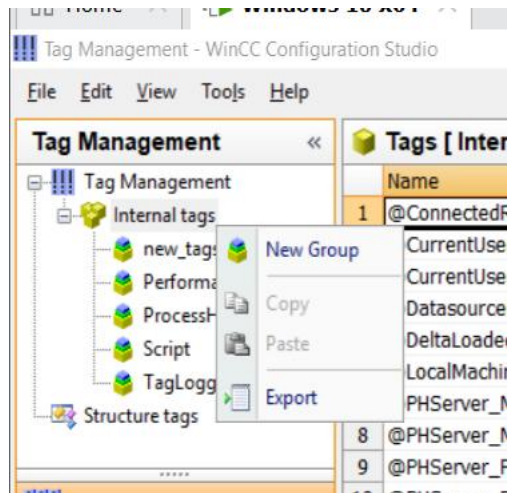


After confirming the project type, the "Create New Project" window appeared. Here, I entered a specific **Project name** to keep the workspace organized. I also specified the **Project path** directory to make sure all upcoming tags, graphic screens, and libraries are stored in the correct local database folder.

Tag Management



After setting up the main project, the next step was to define the variables that would handle the system data. I opened the **Tag Management** editor from the WinCC Explorer tree navigation to start creating the necessary tags.



To keep the workspace structured and clean, I right clicked on **Internal Tags** and created a **New Group** named "new_tags". This helps in separating different process variables as the project grows.

Initially, I started working with a set of 4 basic tags to establish the baseline of the system: **Pressure**, **Status**, **Temperature**, and **Total**.

Tags [new_tags]					
	Name	Comment	Value	Data type	Length
1	Pressure		0	Floating-point number 64-bit IEEE 754	
2	Status		0	Binary Tag	
3	Temperature		0	Signed 8-bit value	
4	Total		0	Unsigned 8-bit value	
5				Signed 16-bit value	
6				Unsigned 16-bit value	
7				Signed 32-bit value	
8				Unsigned 32-bit value	
9				Floating-point number 32-bit IEEE 754	
10				Floating-point number 64-bit IEEE 754	
11				Text tag 8-bit character set	
12				Text tag 16-bit character set	
13				Raw Data Type	
				Text reference	
				Date/Time	

While WinCC supports various data formats like Binary, Integer, or Text, I specifically configured the **Data Type** for these variables as **Floating-point number 64-bit IEEE 754**. I chose this high-precision float type to accurately mimic real-world industrial process data, where analog values from sensors require detailed decimal precision rather than simple whole numbers.

Tag Management - WinCC Configuration Studio

File Edit View Tools Help

Tag Management <<

Tags [new_tags] Find

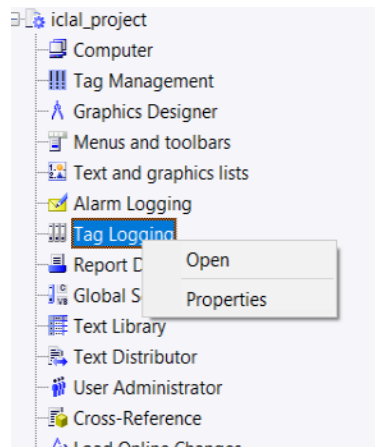
	Name	Comment	Value	Data type	Length	Format adaptation	Connection	Group	Address
1	Pressure		0	Floating-point number 64-bit IEEE	8		Internal tags	new_tags	
2	Status		0	Floating-point number 64-bit IEEE	8		Internal tags	new_tags	
3	Temperature		0	Floating-point number 64-bit IEEE	8		Internal tags	new_tags	
4	Total		0	Floating-point number 64-bit IEEE	8		Internal tags	new_tags	
5									
6									
7									
8									

Tag Management

- Internal tags
 - new_tags
 - Performance
 - ProcessHistorian
 - Script
 - TagLoggingRt
 - Structure tags

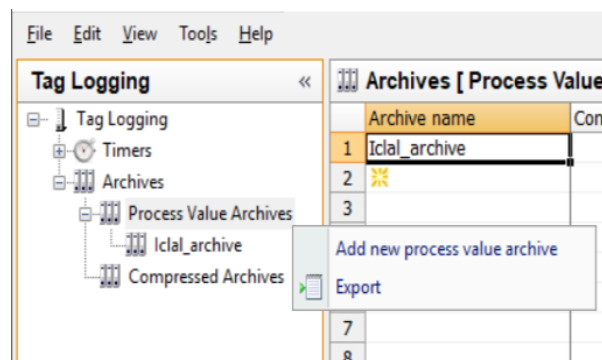
While I began with these 4 core internal tags for development, this list was later expanded under the **Analyzing Multi-Cycle Archiving**. Increasing the number of tags and defining their specific update cycles allowed the system to handle historical data logging and track real-time process changes more efficiently.

Tag Logging and Data Archiving



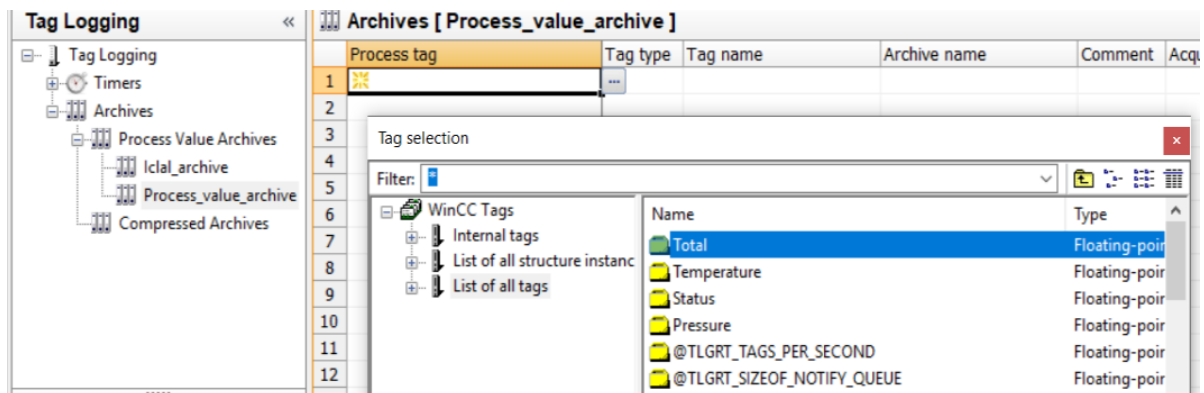
To track historical process trends and manage data storage, I configured the historical data recording system. I navigated back to the project tree and opened the **Tag Logging** editor to set up the data archives.

I right clicked on **Tag Logging** in the WinCC Explorer navigation pane and selected **Open** to access the configuration studio interface.



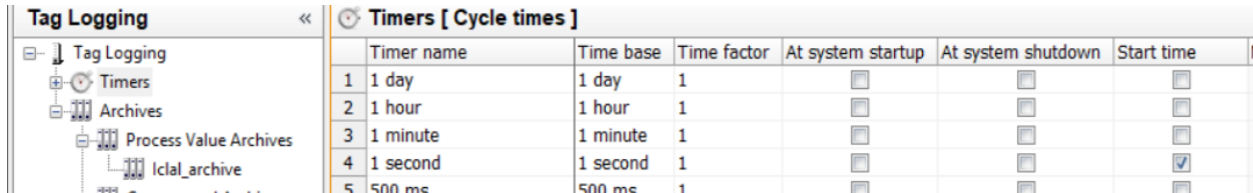
Inside the Tag Logging editor, I set up a dedicated archive directory. As shown in the configuration tree, I established a process value archive named **Process_value_archive** (under the "Process Value Archives" folder). This step is essential for defining where and how the system stores continuous sensor data over long periods.

Within the archive settings, I opened the **Tag selection** window to link our previously created variables to the logging system. I filtered through the WinCC internal tags and selected the process variables—such as **Total**, **Temperature**, **Status**, and **Pressure**—to map them directly into the archive database. This ensures that every fluctuation in these specific data points is securely captured, timestamped, and made available for future analysis or trend screens.



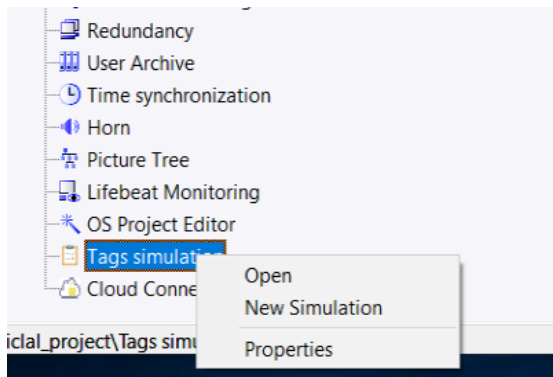
Tag Logging - Simulation

To ensure the system updates data precisely and behaves correctly before connecting to a live PLC, I configured the archiving intervals and set up a dynamic data simulation environment.

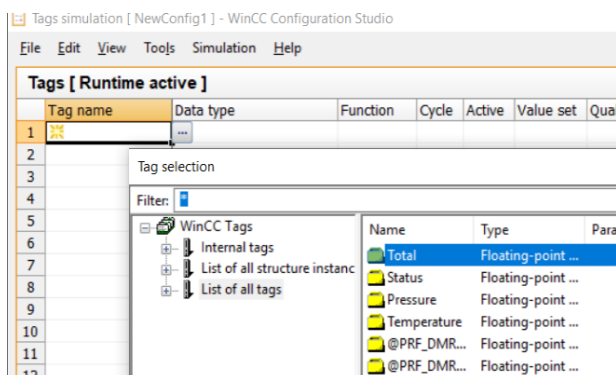


	Timer name	Time base	Time factor	At system startup	At system shutdown	Start time
1	1 day	1 day	1	☐	☐	☐
2	1 hour	1 hour	1	☐	☐	☐
3	1 minute	1 minute	1	☐	☐	☐
4	1 second	1 second	1	☐	☐	☒
5	500 ms	500 ms	1	☐	☐	☐

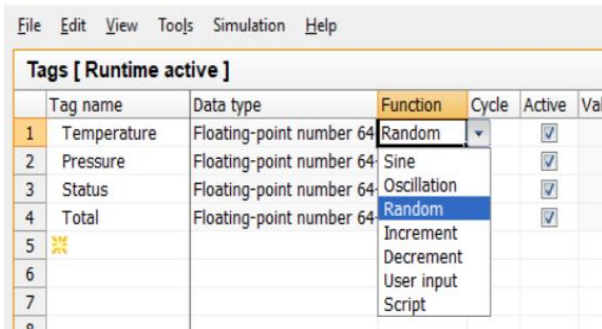
Within the **Tag Logging** properties under the **Timers** section, I analyzed the preset cycle times. For standard real-time monitoring and archiving, the system provides several built-in options like days, hours, minutes, or milliseconds. I selected the **1 second** cycle time as the primary trigger interval to ensure high-resolution data logging for critical process values.



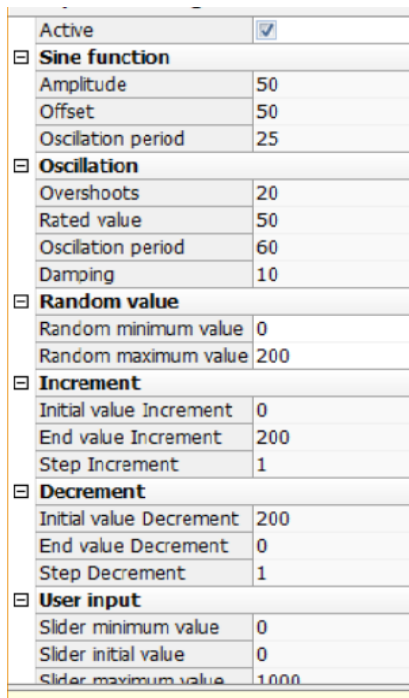
Since physical hardware was not yet linked, I utilized WinCC's built-in simulation utility to generate mock data. I clicked on **Tags simulation** in the project navigation tree and selected **New Simulation** to initialize the simulation workspace.



In the "Tags [Runtime active]" configuration grid, I opened the **Tag selection** window. From the internal tags' directory, I added our core variables—**Total**, **Temperature**, **Status**, and **Pressure**—into the simulation interface to assign them active runtime roles.



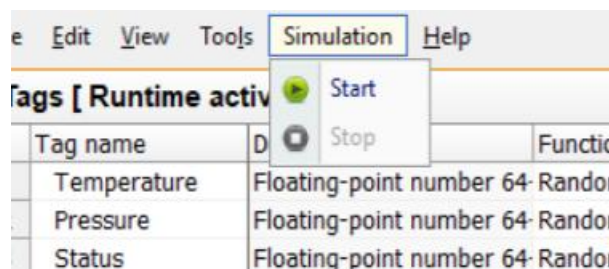
To mimic authentic field signals, I assigned specific behavior types to each tag using the **Function** dropdown menu. I configured them to generate a **Random** value pattern, which closely reflects unpredictable fluctuating sensor readings in a real industrial facility.



To make the simulated signals behave even more like actual factory floor sensors, I adjusted the properties for each simulation function type in the configuration panel. This allowed me to customize the range and pattern of the data generated for each tag.

For variables utilizing the Random value function, I set the Random minimum value to 0 and the Random maximum value to 200. This created a realistic boundary for values like pressure or temperature, preventing the simulation from spiking into impossible or out-of-bound numbers.

For tags tracking cyclical behaviors, the Sine function properties were adjusted with an Amplitude of 50, an Offset of 50, and an Oscillation period of 25. Additionally, the Oscillation parameters were tuned with a Rated value of 50 and specific Damping and Overshoot settings to simulate the physical stabilization of a control loop or a valve adjusting over time.

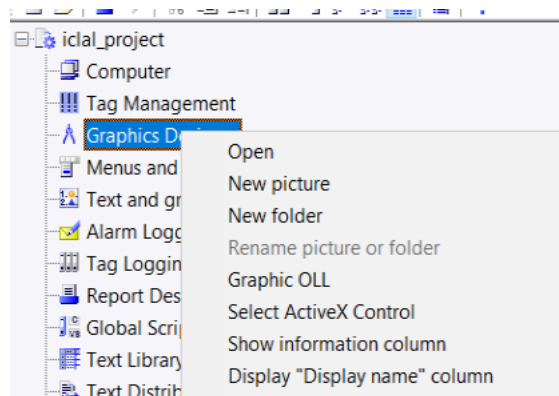


Finally, I went to the top menu, clicked on **Simulation**, and selected **Start**.

File Edit View Tools Simulation Help

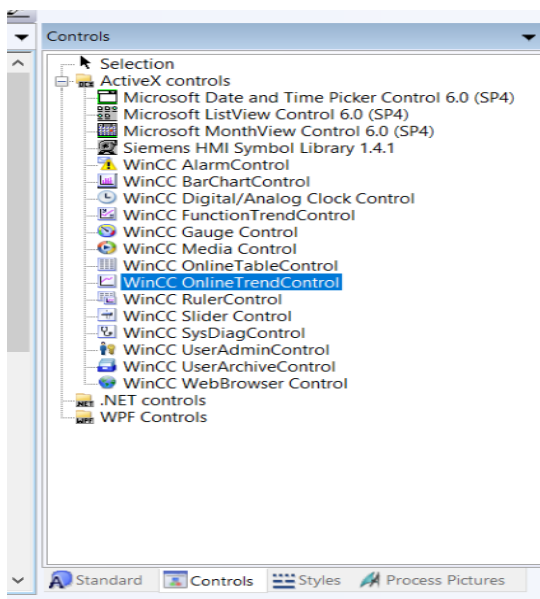
Tags [Simulation active]									Find
	Tag name	Data type	Function	Cycle	Active	Value set	Quality set	Current value	Time
1	Temperature	Floating-point number 64-Random	1		☒	16.669209875 0x60 - uncertain - simulated value		74.6116519669179	8/28,
2	Pressure	Floating-point number 64-Random	1		☒	184.27686391 0x60 - uncertain - simulated value		64.9006622516556	8/28,
3	Status	Floating-point number 64-Random	1		☒	149.38810388 0x60 - uncertain - simulated value		45.3199865718558	8/28,
4	Total	Floating-point number 64-Random	1		☒	90.041810357 0x60 - uncertain - simulated value		85.2626117740409	8/28,
5									

The dashboard immediately began updating, showing dynamic changes in the **Current value** column with a status marked as "simulated value". This verified that the project infrastructure was perfectly ready to feed live-updating data into trends and screen objects. HMI Screens and Real-Time Trend Visualization

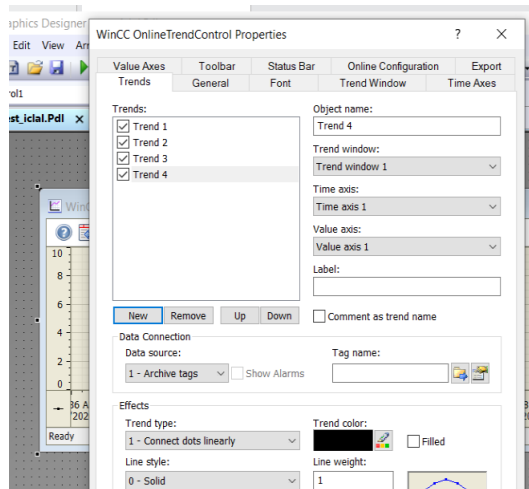


With the internal simulated data running smoothly in the background, I proceeded to create the Human-Machine Interface (HMI) layout to visually monitor these variables over time.

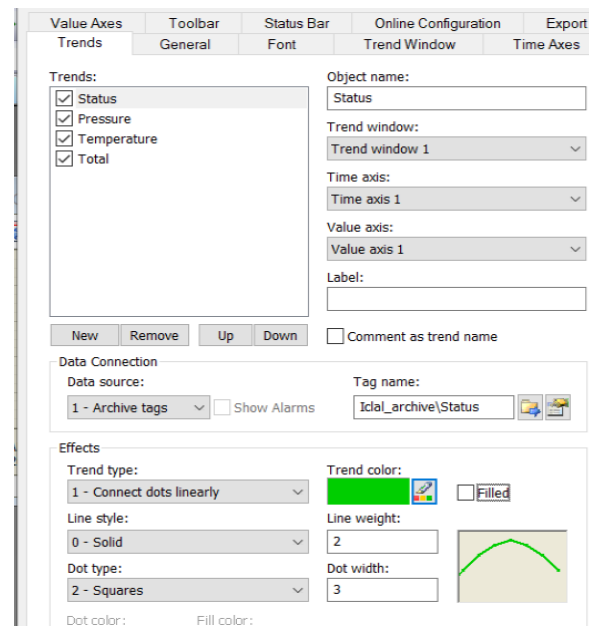
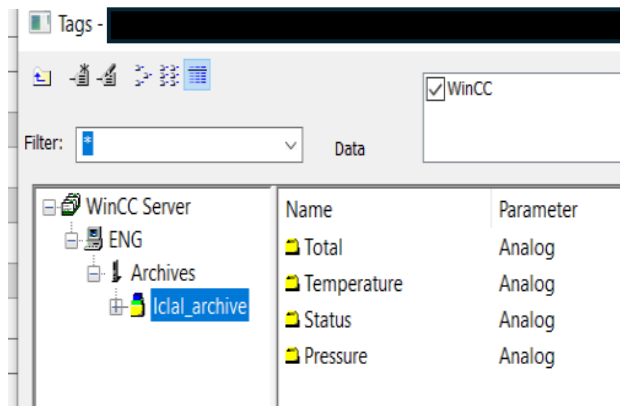
I went to the WinCC Explorer navigation pane, right clicked on **Graphics Designer**, and selected **New picture** to create a blank interface window.



Inside the Graphics Designer workspace, I opened the "Controls" palette and browsed through the available ActiveX controls. I selected and dragged the **WinCC OnlineTrendControl** onto the canvas. This element is specifically built to render live and historical archive logs as continuous graphical lines.

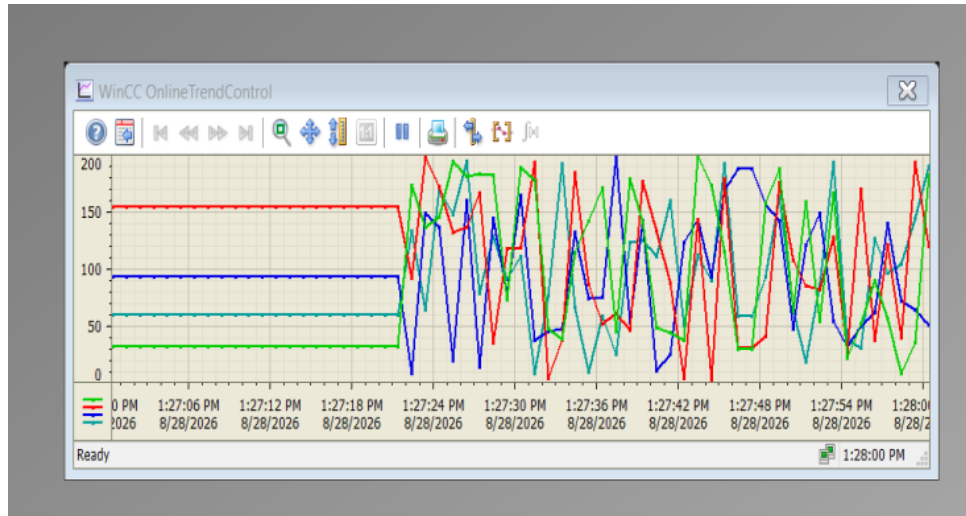


I opened the properties window for the OnlineTrendControl component to link our database to the chart. Under the **Trends** tab, I added four distinct trend entries. I renamed them from their generic names (Trend 1, Trend 2, etc.) to match our exact variables: **Status**, **Pressure**, **Temperature**, and **Total**.

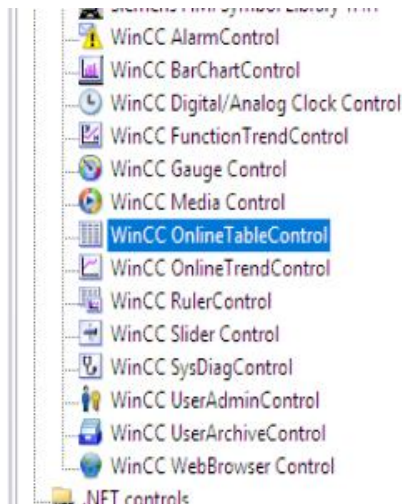


Under the *Data Connection* settings, I changed the *Data Source Field* to **1 - Archive tags**. Using the tag selection dialog, I navigated into the **Iclal_archive** folder located on the WinCC server hierarchy. From there, I mapped each trend object to its respective analog archive tag (e.g., linking the Status trend to Iclal_archive\Status).

To ensure clarity on the final screen, I also customized individual styling features like setting a specific **Trend color** (e.g., green for Status) and choosing the line weight.

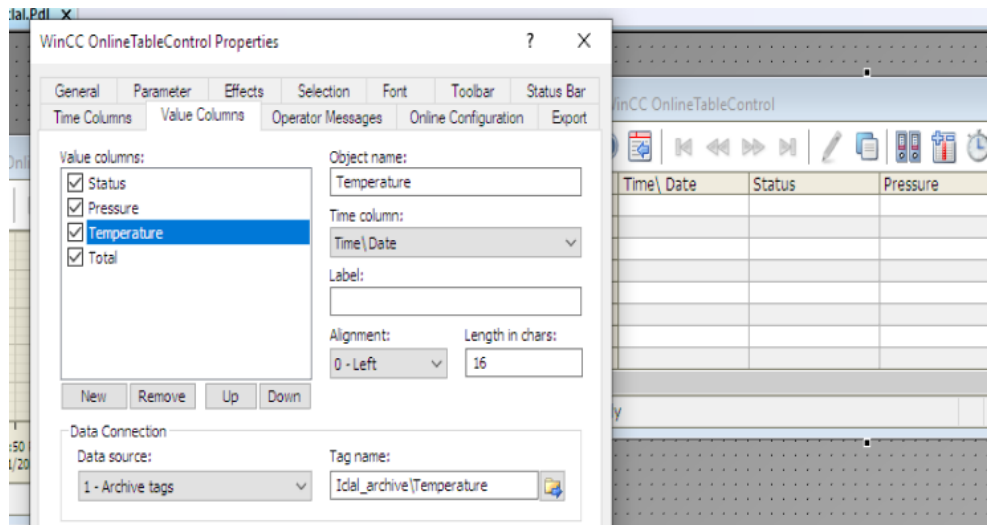


Once configuration was complete, I launched the WinCC Runtime environment. As shown in the final display graph, the **WinCC OnlineTrendControl** began plotting the simulated values instantly. The chart beautifully visualizes the transition from flat baseline numbers into highly fluctuating, dynamic real-time data waves, confirming that the logging intervals, simulation parameters, and graphic objects are fully synchronized.

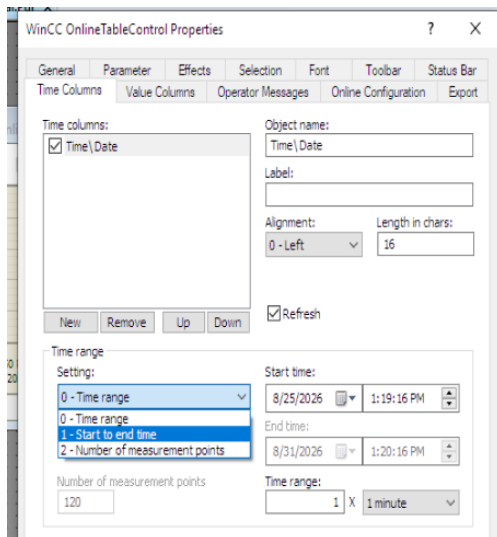


In addition to visual trend charts, industrial SCADA systems require raw numerical logging to provide operators with precise, timestamped event data. To implement this, I integrated a tabular reporting element on the HMI screen.

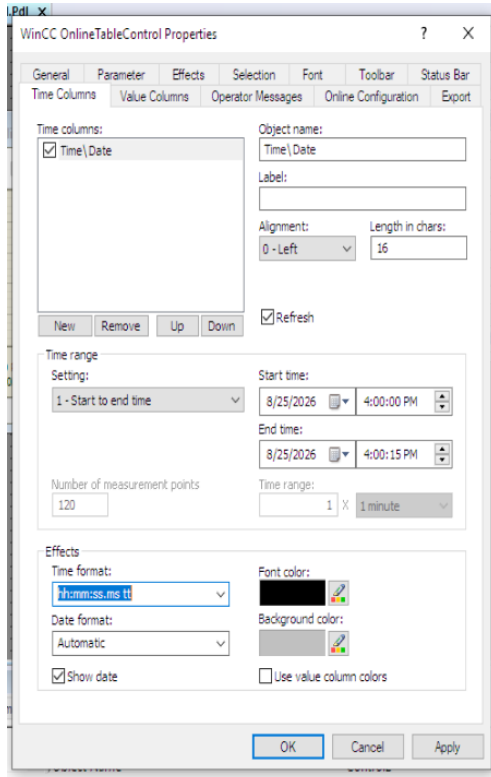
Within the Graphics Designer workspace, I selected the **WinCC OnlineTableControl** from the ActiveX controls menu and dragged it onto the user screen. This component allows archived values to be displayed in a structured spreadsheet format during runtime.



I opened the table properties dialog box and navigated to the **Value Columns** tab. Here, I added four custom columns named **Status**, **Pressure**, **Temperature**, and **Total**. Like the trend configuration, I mapped each column's *Data source* to **1 - Archive tags** and linked them directly to their corresponding database files inside the **Iclal_archive** folder (e.g., matching the Temperature column to Iclal_archive\Temperature).

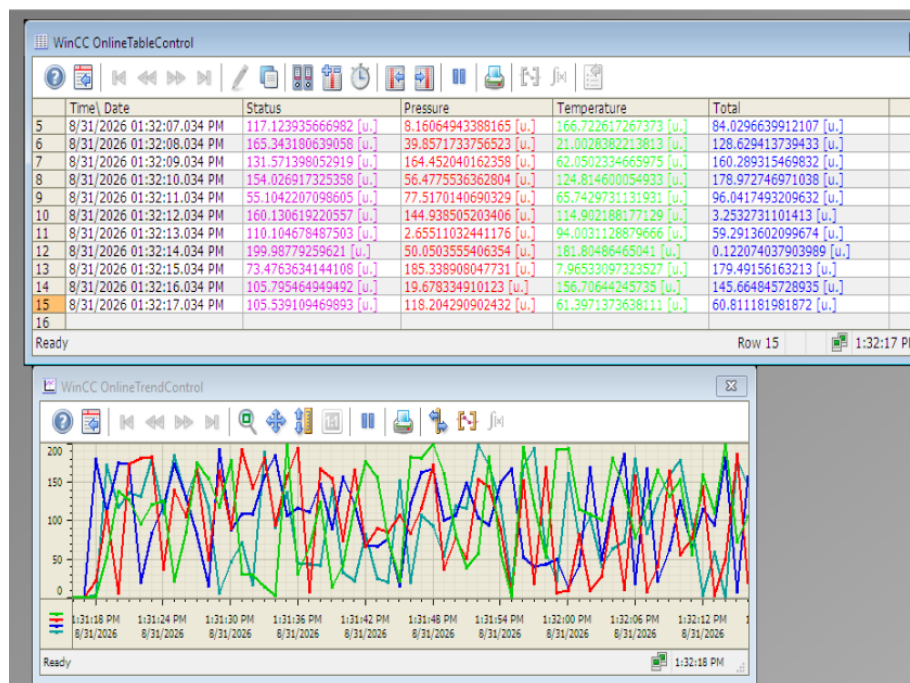


To define how historical data is filtered, I switched to the **Time Columns** tab. Under the *Time range* settings, I changed the *setting* dropdown mode from **0 - Time range** to **1 - Start to end time**. This adjustment allows the system to display records locked between specific timestamps.



I then configured a precise test boundary from **4:00:00 PM to 4:00:15 PM** and set the time format display style to hh:mm:ss.ms.tt to capture values down to the exact millisecond.

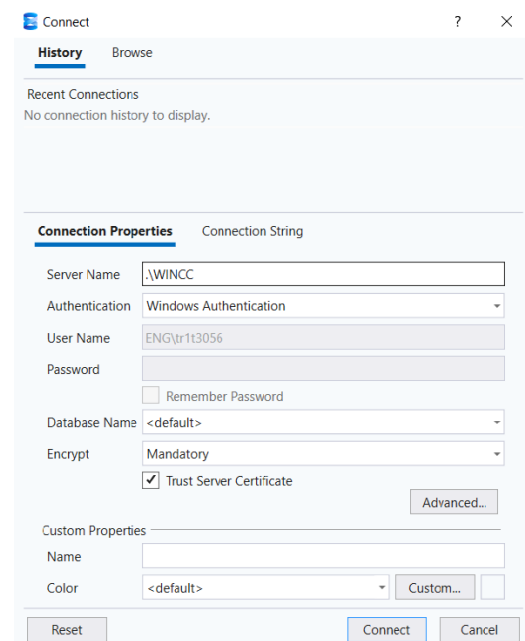
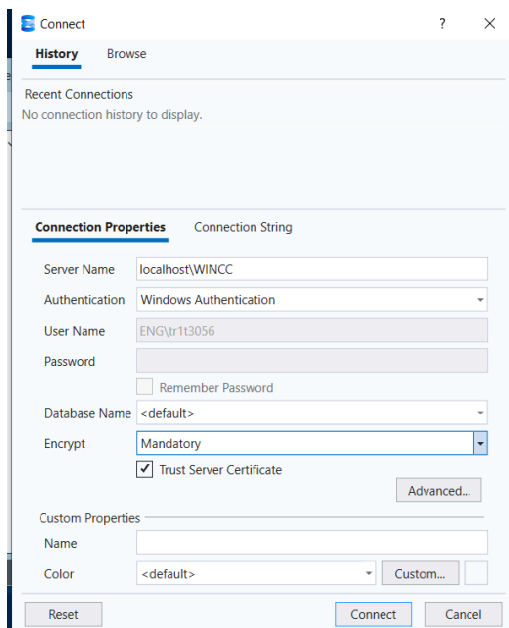
After completing the layout, I verified the configuration in WinCC Runtime. WinCC **OnlineTableControl** successfully generated an active data grid alongside our trend chart. Each row tracks the changing parameters second-by-second, clearly displaying the exact timestamp, date, and numerical float values. The status suffix [u.] indicates simulated/uncertain values, confirming that our background simulation engine is perfectly logging numerical history into the tabular database.



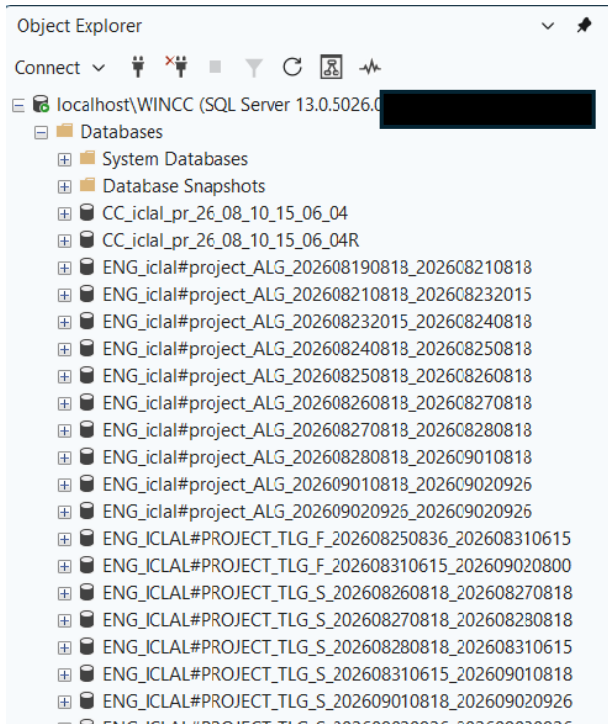
Database Management with SQL Server Management Studio (SSMS)



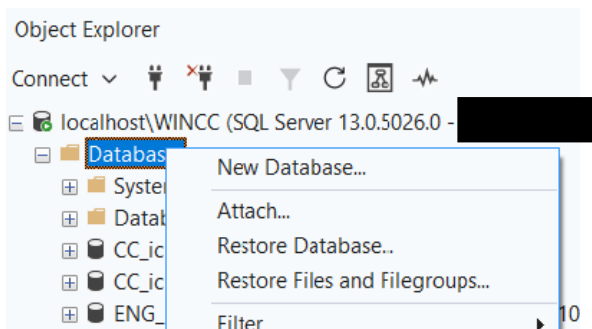
Siemens WinCC utilizes a Microsoft SQL Server infrastructure in the background to log, archive, and manage all runtime data, trends, and system configurations. To inspect how WinCC interacts with the backend database and to create a custom standalone database for manufacturing data, I utilized **SQL Server Management Studio 22 (SSMS)**.



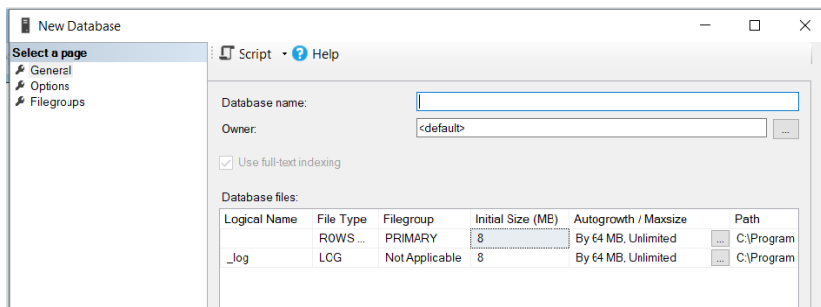
First, I launched SSMS. In the connection window, I set the *Server Name* to **localhost\WINCC or .\WINCC** to target the local SQL instance installed by the WinCC setup. I used **Windows Authentication** to log in securely using my current workstation credentials. Additionally, it was critical to adjust the Connection Properties by checking the **Trust Server Certificate** box and setting the *Encrypt* option to *Mandatory*. This step ensures a secure connection to the local instance without encountering driver-level security blocks.



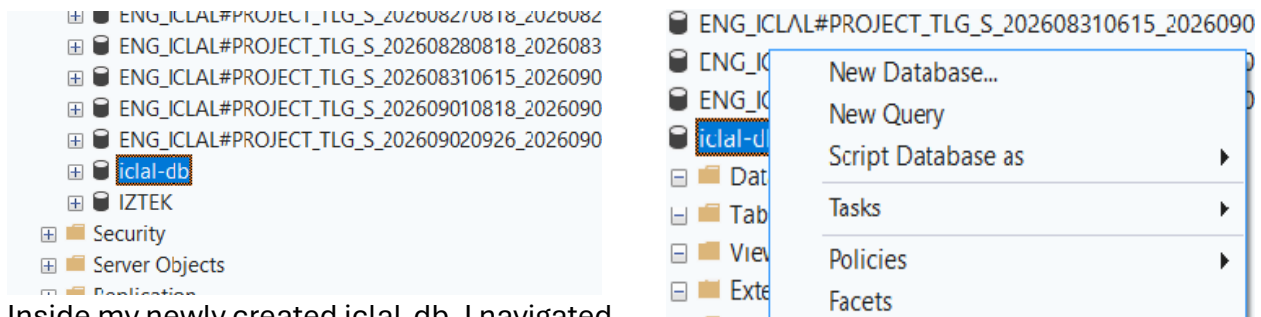
Once connected, I expanded the **Databases** directory in the *Object Explorer*. Here, I observed the internal database files automatically generated by WinCC for my active project, including configuration databases (prefixed with CC_iclal_pr...), alarm logging tables (..._ALG...), and the tag logging archives (..._TLG...) created in the previous steps.



To store independent production data or custom logs, I right clicked on the *Databases* folder and selected **New Database....**



In the configuration dialog, I named the new database **iclal-db** and kept the default initial file size and autogrowth settings to let the primary and log files expand dynamically.

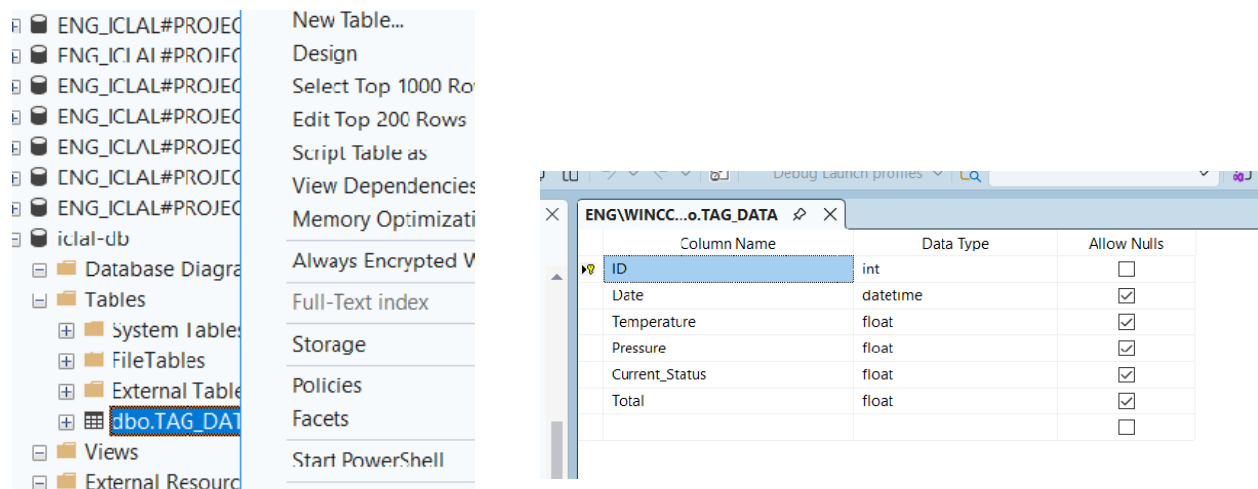


Inside my newly created iclal-db, I navigated to the Tables folder, right-clicked, and selected New Table... to define a structured storage scheme. I designed a table named dbo.TAG_DATA with columns specifically tailored to hold our process parameters:

ID (int) – Acts as a unique key for indexing rows.

Date (datetime) – Stores the precise timestamp of when data is captured.

Temperature (float), Pressure (float), Current_Status (float), and Total (float) – Set to the float data type to match the 64-bit precision values we simulated and archived inside WinCC Explorer.



The target architecture requires the ID column to serve as a unique identifier and key constraint, while allowing numerical process variables to store empty values (NULL) if a sensor disconnects. The following **Data Definition Language (DDL)** script was executed in SSMS to construct the dbo.TAG_DATA table exactly as structured in the system.

The complete SQL query script used for creating this table structure can also be accessed directly via the [GitHub link](#) provided.

Once the database layout was ready, I established a bridge between the WinCC SCADA platform and SQL Server. To do this, I developed a custom **global VBScript** inside WinCC. The script triggers cyclically or upon value change, automatically opening an **ADODB.Connection** to the local SQL instance, pulling the simulated tag values (Temperature, Pressure, etc.), and executing an INSERT INTO SQL command to push the live data records directly into the table.

As demonstrated in the runtime results panel, the table dynamically populates with second-by-second high-precision float logs matching the internal WinCC parameters perfectly.

SQLQuery3.s...t3056 (98)* ENG\WINCC.i...bo.TAG_DATA

```

1 SELECT [ID]
2       , [Date]
3       , [Temperature]
4       , [Pressure]
5       , [Current_Status]
6       , [Total]
7 FROM [iclal-db].[dbo].[TAG_DATA]
8

```

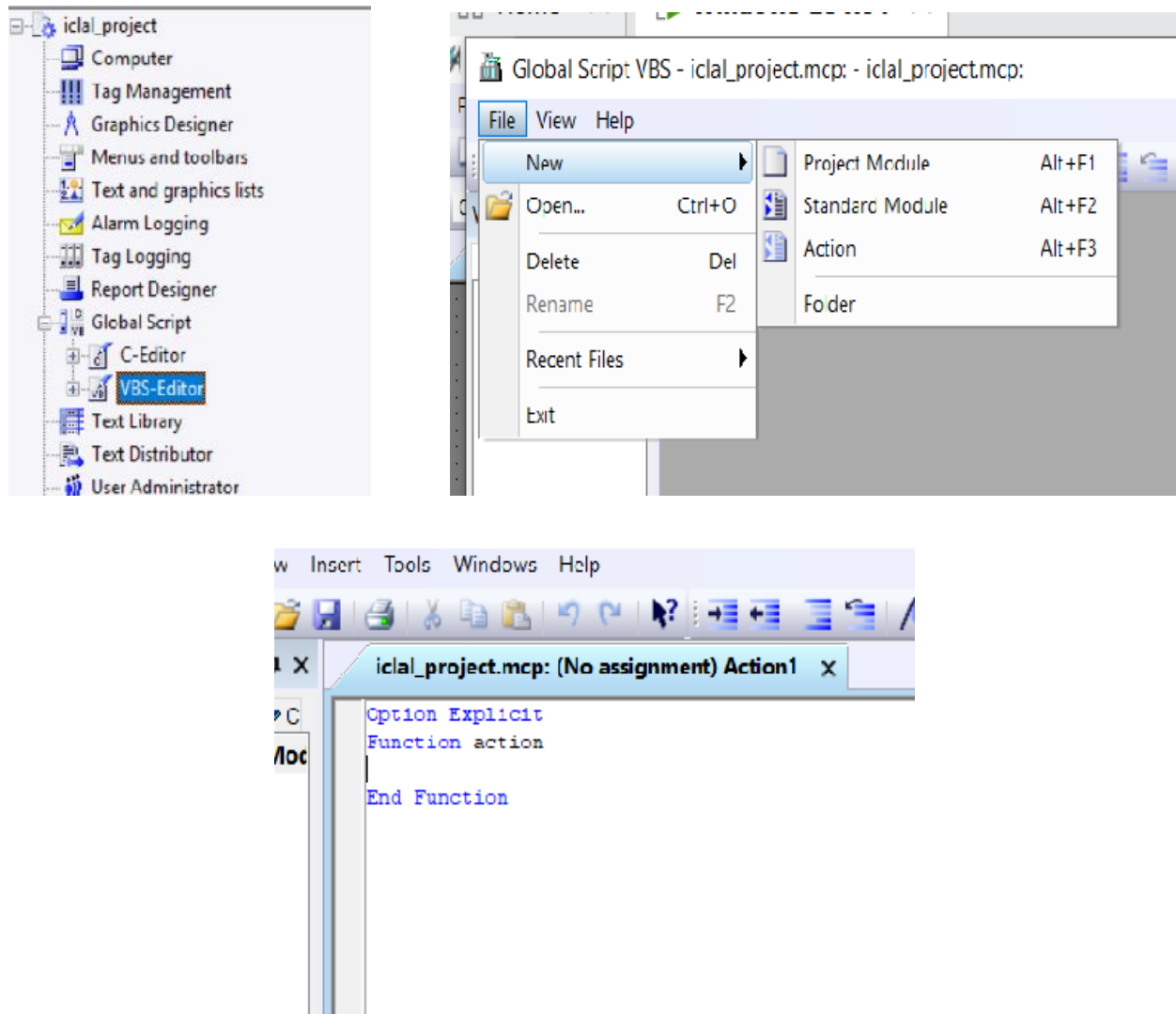
90 % No issues found

Results Messages

ID	Date	Temperature	Pressure	Current_Status	Total
133775	2026-08-25 15:59:58.000	183.94116031373	127.506332590716	47.9934080019532	119.168675801874
133776	2026-08-25 15:59:59.000	49.0249336222419	5.36515396588031	70.3817865535447	57.8997161778619
133777	2026-08-25 16:00:00.000	93.6368907742546	53.5966063417463	86.1559596545305	9.21048615985595
133778	2026-08-25 16:00:01.000	3.48521378215888	153.404339732047	136.576433606983	94.8088015381329
133779	2026-08-25 16:00:02.000	177.233191930906	117.50846888638	57.3870052186651	85.5922116763817
133780	2026-08-25 16:00:03.000	167.509994811853	192.016357921079	98.2757042146062	168.578142643513
133781	2026-08-25 16:00:04.000	105.166783654286	8.6794640949736	190.997039704581	167.320780053102
133782	2026-08-25 16:00:05.000	162.968840601825	18.8421277504807	172.567622913297	97.1465193639943
133783	2026-08-25 16:00:06.000	14.996795556505	140.208136234626	170.354319895016	36.3231299783319
133784	2026-08-25 16:00:07.000	148.002563554796	47.7858821375164	53.2425916318247	194.586016418958
133785	2026-08-25 16:00:08.000	180.944242683187	40.2478102969451	158.547419660024	160.093997009186
133786	2026-08-25 16:00:09.000	49.0188299203467	112.875759147923	198.071230201117	6.23187963499863
133787	2026-08-25 16:00:10.000	8.37427900021363	168.437757499924	149.538355662709	39.3627735221412

As shown in the SSMS query results window, executing a SELECT query confirms that the **VBScript successfully pushes all dataset fields straight into the dbo.TAG_DATA table automatically**. The records show a perfect chronological sequence with zero skipped logs, verifying that the dynamic industrial data flows automatically from the WinCC monitoring platform into the persistent SQL storage layer.

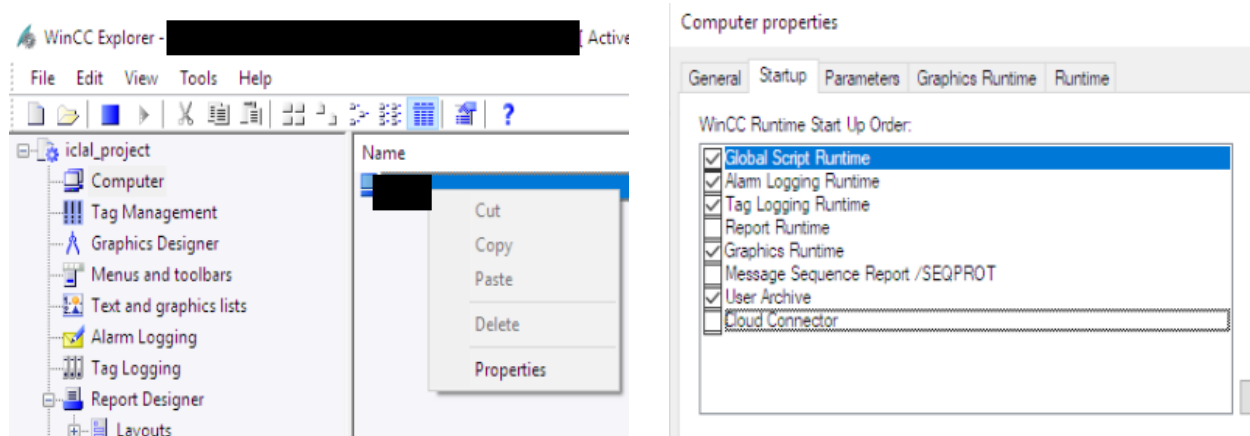
Global Script



To automate the data exchange between the SCADA runtime tags and the Microsoft SQL Server database, I developed a background script using the WinCC scripting architecture. Instead of binding the script to a specific graphic screen, I implemented it as a global routine to ensure it runs continuously in the background.

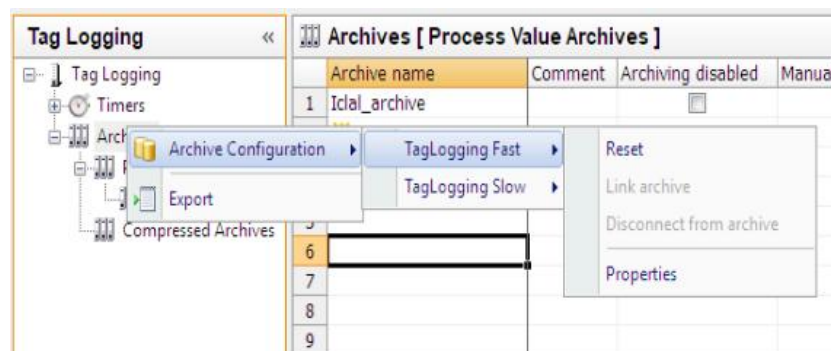
The full automated background logic, connection strings, and the detailed open-source VBScript architecture implemented for this continuous data transaction can be reviewed via the [GitHub repository link](#) provided.

Configurations

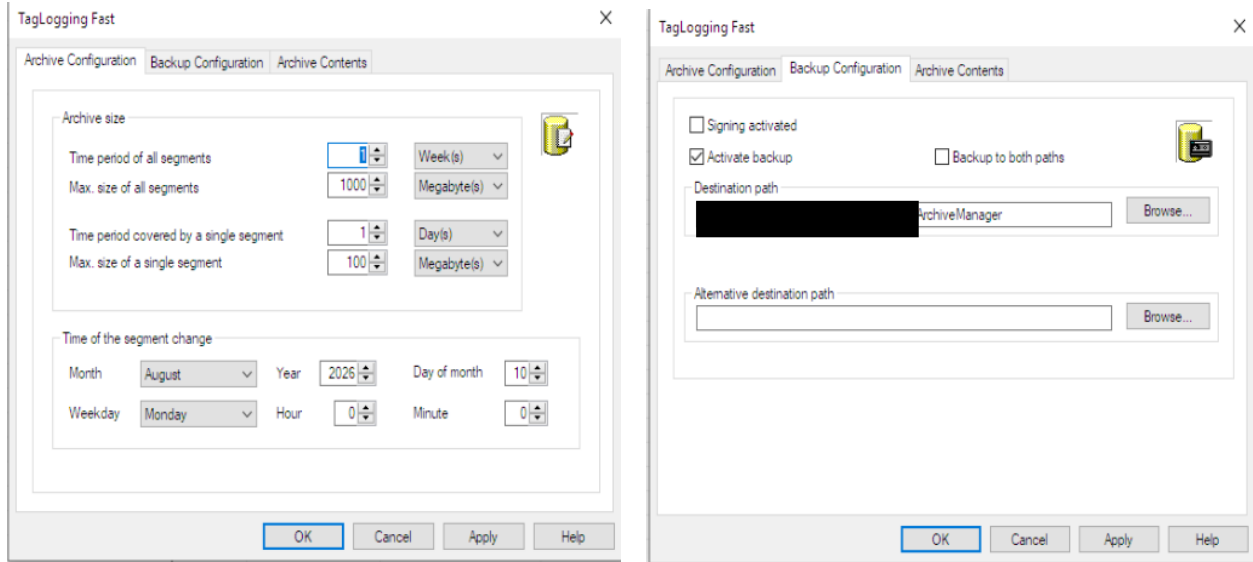


To ensure that the created global scripts, tag archives, and database interactions execute correctly when the SCADA system starts, I finalized the project-wide system parameters and database backup configurations.

First, I went to WinCC Explorer, clicked on the computer node, and selected **Properties**. In the *Computer Properties* dialog box, I navigated to the **Startup** tab. Under the *WinCC Runtime Start Up Order* list, I checked the box for **Global Script Runtime**. This step is vital; if it is left unchecked, the background VBScript task that automates the data flow to the SQL database will not be executed when the SCADA screen goes online.



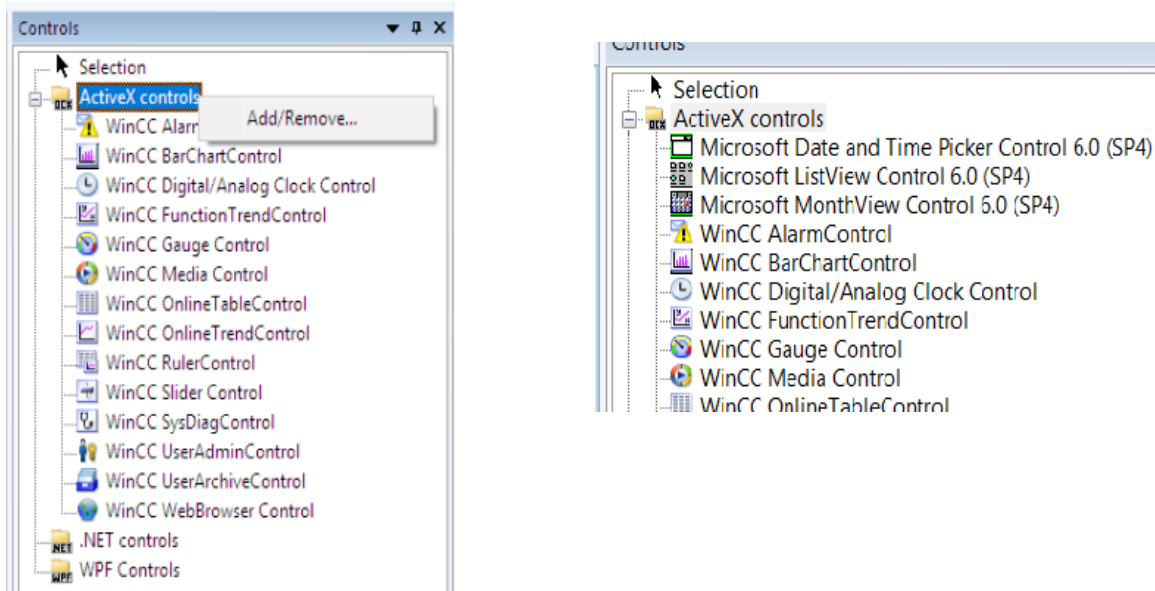
Next, I adjusted the storage properties of the tag logging framework. In the **Tag Logging** editor, I right clicked on TagLogging Fast under the archive configuration tree to open its dedicated setup dashboard.



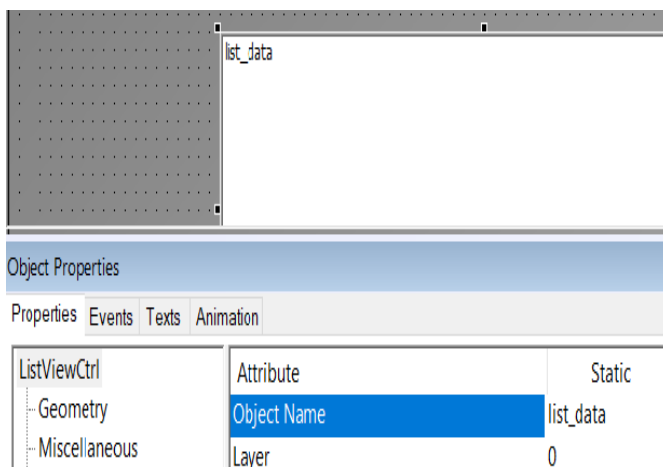
Under the *Archive Configuration* tab, I set the total time period of all segments to **1 week** and limited the max size of all segments to **1000 Megabytes** to keep the local disk usage optimized.

To protect historical data against local data corruption, I switched to the **Backup Configuration** tab within the same properties window. I checked the **Activate backup** option. Then, I mapped the primary **Destination path** folder directly to my desktop project files under ...\\icla1_project\\ArchiveManager. This setup commands WinCC to automatically extract, compress, and mirror historical runtime data directly into a secure directory alongside the standard SQL tables.

External Database Filtering (iclal-db)

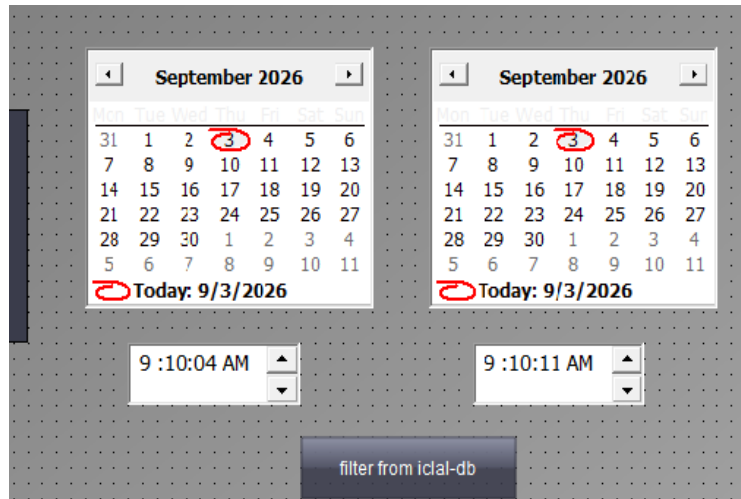


To create an interactive and operator-friendly query panel within the **iclal_test.pdl** screen, I integrated specialized external libraries and controls that are not available by default in the standard WinCC toolkit. Inside the Graphics Designer workspace, I navigated to the **Controls** palette, clicked on *ActiveX controls*, and selected **Add/Remove....** To build the calendar and time interfaces, I registered the Windows system libraries: **Microsoft Date and Time Picker Control 6.0 (SP4)** and **Microsoft MonthView Control 5.0 (SP4)**.

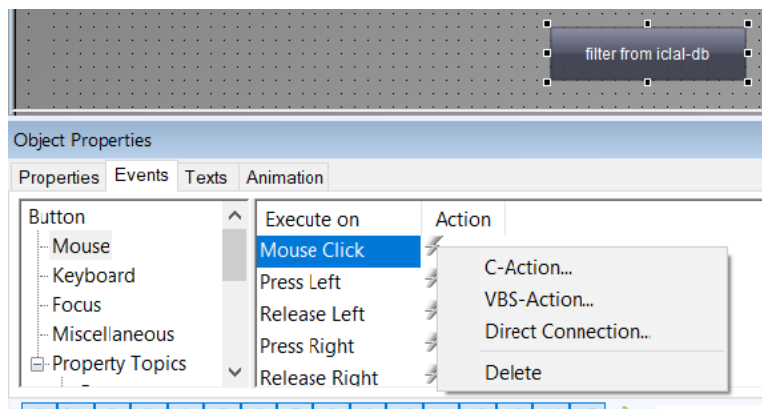


Once registered, I dragged these custom controls onto the canvas to construct a unified query criteria block. This includes two graphical calendar elements for selecting date thresholds and two dedicated spinner input boxes for defining the time parameters down to the exact second. Furthermore, I integrated a **ListViewCtrl** object from the ActiveX menu, naming it **list_data** inside its geometric object properties to

serve as a clean container for displaying data structures.



To trigger the data transactions, I placed a custom button labeled **filter from iclal-db**. In the *Object Properties* explorer, I navigated to *Events* -> *Button* -> *Mouse*, right-clicked on the **Mouse Click** event trigger, and initialized a blank **VBS-Action...** editor. Within this handler, I developed the logic that reads the values from the calendar pickers, formats them into a database-compliant string, and injects them into the SQL search parameters.



As verified in the active WinCC Runtime screen window, the complex filtering engine executes successfully. Operators can check explicit parameter boxes on the left panel (such as checking Status and Pressure while leaving Temperature and Total unchecked). Clicking *filter from iclal-db* commands the system to open an ADODB connection pipeline, pull records restricted strictly between the selected window (e.g., 8/25/2026 4:00:00 PM to 4:00:08 PM), and dynamically populate the reporting grid with the precise timestamped float values.

WinCC-Runtime -

☒ Status
☒ Pressure
☐ Temperature
☐ Total

August 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

August 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

4 :00:00 PM

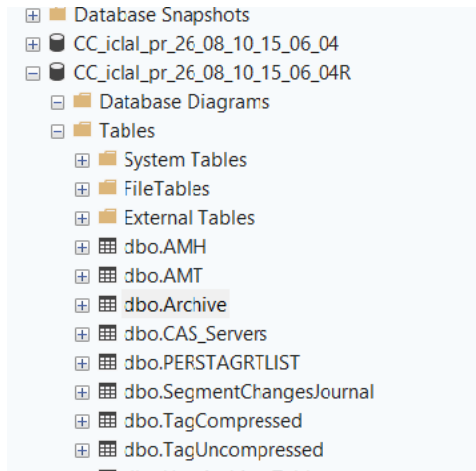
4 :00:08 PM

filter from lclal-db

Date	Pressure	Current_Status
8/25/2026 4:00:00 PM	53.5966063417463	86.1659596545305
8/25/2026 4:00:01 PM	153.404339732047	136.576433606983
8/25/2026 4:00:02 PM	117.50846888638	57.3870052186651
8/25/2026 4:00:03 PM	192.016357921079	98.2757042146062
8/25/2026 4:00:04 PM	8.6794640949736	190.997039704581
8/25/2026 4:00:05 PM	18.8421277504807	172.667622913297
8/25/2026 4:00:06 PM	140.208136234626	170.354319895016
8/25/2026 4:00:07 PM	47.7858821375164	53.2425916318247
8/25/2026 4:00:08 PM	40.2478102969451	158.647419660024

The complete, updated VBScript code blocks containing the ActiveX component interaction handles, dynamic SQL query formatting rules, and dataset mapping functions can be reviewed in detail via the provided ***Github Link for tag choice*** and ***Github Link for filtering from lclal-db***.

Investigating Internal WinCC Archives and Implementing Automated Tag Discovery



To expand the application's runtime capabilities and eliminate hardcoded variable definitions, I investigated the internal database architecture generated automatically by Siemens WinCC. Industrial monitoring systems must dynamically adapt when engineers add or remove field sensors, making runtime tag discovery a crucial design step.

I used SQL Server Management Studio (SSMS) to inspect the active runtime database, specifically targeting the project configuration folder CC_iclal_pr_26_08_10_15_06_04R. I executed a structured query to audit the core system catalogs:

SQLQuery7.s...1t3056 (83)) X SQLQuery6.sq...1t3056 (105))*

```

1  SELECT TOP (1000) [ValueID]
2    , [ValueName]
3    , [LocaleID]
4    , [CompPrecision]
5    , [CompressionMode]
6    , [VarType]
7    , [Flags]
8    , [CtrF]
9    , [CtrS]
10   , [ID]
11   , [LastModification]
12 FROM [CC_iclal_pr_26_08_10_15_06_04R].[dbo].[Archive]
13

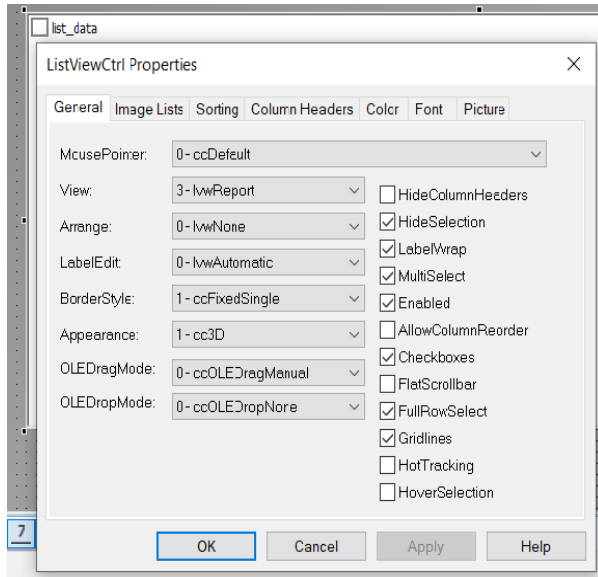
```

90 % No issues found

Results Messages

	ValueID	ValueName	LocaleID	CompPrecision	CompressionMode	VarType	Flags	CtrF	CtrS	ID	LastModification
1	1	lclal_archive\Pressure	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
2	2	lclal_archive>Status	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
3	3	lclal_archive\Temperature	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
4	4	lclal_archive\Total	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
5	11	lclal_archive\deneme	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
6	13	lclal_archive\new_test	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000
7	14	lclal_archive\lclal_test	0	0.2	5	5	1	2	0	0	2026-09-08 07:38:21.000

As illustrated in the query results panel, the internal WinCC registry maps every single logged variable to a unique transactional token (ValueID). For example, Iclal_archive\Pressure is indexed as ValueID 1, Status as 2, and the newly implemented Iclal_test variable as 14.



To present this discovered structural metadata dynamically to the operator, I updated the **ListViewCtrl** object properties within Graphics Designer. In the *ListViewCtrl Properties* dialog box under the *General* tab, I set the *View* configuration to **3 - lwReport** to enable a clean tabular database view. Crucially, I checked the **Checkboxes** option to provide a selection interface, and enabled **FullRowSelect** and **GridLines** to ensure high visibility during runtime interactions.

To automate the user panel, I implemented a dedicated control button labeled **tag_choice**. I mapped a VBScript routine to its click event that automatically establishes an internal OLE DB/ADO connection string directly to the WinCC runtime instance. Instead of forcing manual text entries, clicking **tag_choice** dynamically executes a backend scan on the `dbo.Archive` table, parses the registered variable definitions, and instantly populates the active ListView tracking grid with the actual live tags and their associated checkbox index keys.

After the active tags are mapped into the tracking list, operators can select precise variables via the checkboxes and hit the **filter from WINCC DB** button. The background execution script scans the selected items, forms a customized conditional query string, filters the local historical records across the selected calendar timelines, and updates the main reporting matrix. As verified in the active screen, selecting specific variables dynamically repopulates the dashboard columns, maintaining automated aggregate metrics like **COUNT**, **AVERAGE**, **MINIMUM**, and **MAXIMUM** tracking at the baseline of the runtime table.

The complete, production-ready VBScript code blocks containing the internal database connection strings, recursive checkbox scanning loops, and dynamic column layout structures can be reviewed in detail via the provided **[GitHub link](#)**.

August 2026

27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

August 2026

27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

4:00:00 PM

4:00:10 PM

tag_choice

filter from WINCC DB

Tag ID	Tag Name
☐ 1	Pressure
☒ 2	Status
☒ 3	Temperature
☒ 4	Total
☐ 5	deneme
☐ 6	new_test
☐ 7	Ical_test

August 2026

27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

August 2026

27	28	29	30	31	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Today: 9/3/2026

4:00:00 PM

4:00:10 PM

tag_choice

filter from WINCC DB

Date / Time	Status	Temperature	Total
☐ 2026-08-25 16:00:00	70.38	49.02	57.90
☐ 2026-08-25 16:00:01	86.17	93.64	9.21
☐ 2026-08-25 16:00:02	136.58	3.49	94.81
☐ 2026-08-25 16:00:03	57.39	177.23	85.59
☐ 2026-08-25 16:00:04	98.28	167.51	168.58
☐ 2026-08-25 16:00:05	191.00	105.17	167.32
☐ 2026-08-25 16:00:06	172.67	162.97	97.15
☐ 2026-08-25 16:00:07	170.35	15.00	36.32
☐ 2026-08-25 16:00:08	53.24	148.00	194.59
☐ 2026-08-25 16:00:09	158.65	180.94	160.09
☐ -----			
☐ COUNT	10	10	10
☐ AVERAGE	119.47	110.30	107.16
☐ MINIMUM	53.24	3.49	9.21
☐ MAXIMUM	191.00	180.94	194.59

Exporting data as .txt form

The screenshot shows a dynamic HMI layout with a data reporting grid. At the top, there are two date/time selection controls set to 4:00:00 PM and 4:00:11 PM, both for 'Today, 9/13/2020'. To the right of these controls are buttons for 'tag_choice' and 'filter from WINCC DB'. Below these is a large 'export' button. The data grid has the following columns: Date / Time, Pressure, Status, Temperature, and Total. It displays 11 rows of data for the date 2026-08-25, followed by summary rows for COUNT, AVERAGE, MINIMUM, and MAXIMUM. The MINIMUM and MAXIMUM rows are highlighted in green and red respectively.

Date / Time	Pressure	Status	Temperature	Total
2026-08-25 16:00:00	5.37	70.38	49.02	57.90
2026-08-25 16:00:01	53.60	86.17	93.64	9.21
2026-08-25 16:00:02	153.40	136.58	3.49	94.81
2026-08-25 16:00:03	117.51	57.39	177.23	85.59
2026-08-25 16:00:04	192.02	98.28	167.51	168.58
2026-08-25 16:00:05	8.68	191.00	105.17	167.32
2026-08-25 16:00:06	18.84	172.67	162.97	97.15
2026-08-25 16:00:07	140.21	170.35	15.00	36.32
2026-08-25 16:00:08	47.79	53.24	148.00	194.59
2026-08-25 16:00:09	40.25	158.65	180.94	160.09
2026-08-25 16:00:10	112.88	198.07	49.02	6.23
COUNT	11	11	11	11
AVERAGE	80.96	126.62	104.73	97.98
MINIMUM	5.37	53.24	3.49	6.23
MAXIMUM	192.02	198.07	180.94	194.59

As shown in the dynamic HMI layout, I implemented an **export** button alongside the reporting grid. Operators can select a specific time range using the calendar controls, filter the data from the SQL Server database, and then initiate the file extraction process with a single click.

The screenshot shows a Notepad window titled 'WinCC_Exported_Data - Notepad' displaying the exported data. The data is identical to the grid shown in the previous screenshot, including the summary rows. The Notepad window is overlaid on the HMI layout, which is partially visible in the background. The HMI layout also shows a 'WinCC_Exported_Data' icon on the left side.

Date / Time	Pressure	Status	Temperature	Total
2026-08-25 16:00:00	5.37	70.38	49.02	57.90
2026-08-25 16:00:01	53.60	86.17	93.64	9.21
2026-08-25 16:00:02	153.40	136.58	3.49	94.81
2026-08-25 16:00:03	117.51	57.39	177.23	85.59
2026-08-25 16:00:04	192.02	98.28	167.51	168.58
2026-08-25 16:00:05	8.68	191.00	105.17	167.32
2026-08-25 16:00:06	18.84	172.67	162.97	97.15
2026-08-25 16:00:07	140.21	170.35	15.00	36.32
2026-08-25 16:00:08	47.79	53.24	148.00	194.59
2026-08-25 16:00:09	40.25	158.65	180.94	160.09
2026-08-25 16:00:10	112.88	198.07	49.02	6.23
COUNT	11	11	11	11
AVERAGE	80.96	126.62	104.73	97.98
MINIMUM	5.37	53.24	3.49	6.23
MAXIMUM	192.02	198.07	180.94	194.59

Before transferring raw rows, the script checks if any data exists in the current grid view. If valid records are present, it loops through the table columns and writes out comma-separated values (CSV style) or tab-separated alignments. Crucially, the script doesn't just copy raw logs; it automatically appends calculated field metrics—including total row **COUNT**, **AVERAGE** trends, **MINIMUM** thresholds, and **MAXIMUM** spikes—directly onto the bottom lines of the text file.

```
rowCount = objTable.Columns.Count

If rowCount = 0 Then
    MsgBox "There is no data to filter!", vbExclamation, "No data"
    Exit Sub
End If

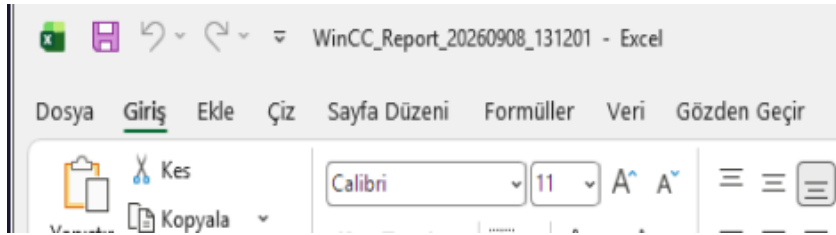
Set wshShell = CreateObject("WScript.Shell")
desktopPath = wshShell.SpecialFolders("Desktop")
filePath = desktopPath & "\WinCC_Exported_Data.txt"

Set fso = CreateObject("Scripting.FileSystemObject")
Set file = fso.CreateTextFile(filePath, True)
```

As verified in the Notepad view screenshot, the automated operation builds a clean, structured .txt manifest directly on the desktop. The file perfectly captures the timestamped parameters (Pressure, Status, Temperature, Total) along with the aggregated analytical calculations at the baseline, proving that the integration provides standalone reporting capabilities independent of the active SCADA software.

Only change will be the file path is Exported_Data.txt. The complete, updated VBScript code blocks containing the file creation routines, loop structures, and statistical calculations can be reviewed in detail via the provided ***Github link***.

Exporting data as .xls form



To expand the reporting capabilities of the SCADA system, I developed an alternative export routine that targets spreadsheet applications. This feature allows management and engineers to review historical logs directly within standard analytical tools like Microsoft Excel without manually converting raw text data.

	A	B	C	D	E
1	EXPORT TIME: 9/8/2026 13:12:01				
2	Date / Time	Pressure	Status	Temperature	Iclal_test
3	2026-09-08 13:04:40	68.95	17.1	130.59	-
4	2026-09-08 13:04:41	93.39	104.03	80.87	-
5	2026-09-08 13:04:42	111.7	131.68	181.82	-
6	2026-09-08 13:04:43	156.18	151.59	96.95	-
7	2026-09-08 13:04:44	41.91	147.96	20.56	-

As shown in the updated HMI screenshot, I enhanced the control panel layout by integrating comprehensive date-time picker controls. Operators can use the dual graphical calendars to lock in an explicit start and end date/time. Once the timeline is set, clicking the custom navigation buttons triggers a background query to filter records from the SQL Server database and refresh the standalone tabular grid alongside the real-time WinCC OnlineTrendControl.

23	2026-09-08 13:05:00	175.57	66.21	69.38	176.83
24	2026-09-08 13:05:01	185.24	15.29	114.93	-
25	2026-09-08 13:05:02	12.36	188.29	93.98	-
26	2026-09-08 13:05:03	44.33	173.43	178.56	-
27	2026-09-08 13:05:04	26.81	139.72	29.69	-
28	-----				
29	COUNT	25	25	25	1
30	AVERAGE	98.28	98.43	92	176.83
31	MINIMUM	3.34	13.67	2.88	176.83
32	MAXIMUM	192.6	188.29	197.23	176.83

To generate native spreadsheet formats programmatically, I wrote an advanced VBScript routine inside the global action manager. The script leverages Component Object Model (COM) automation by creating an **Excel Application object** (Set objExcel = CreateObject("Excel.Application")). It commands the application to run invisibly in the background, spawns a new workbook (objExcel.Workbooks.Add), and targets the primary worksheet layer to inject our dataset.

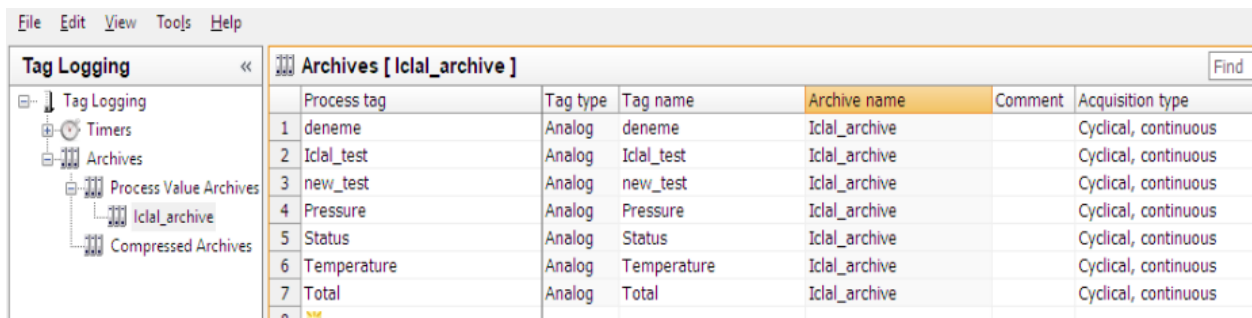
As shown in the Microsoft Excel runtime view, the script outputs a perfectly formatted WinCC_Report_20260908_131201 file. The spreadsheet successfully captures the structured logs, handles cell grids neatly, and positions the mathematical analytics at the bottom. This layout ensures that extensive manufacturing logs can be shared, filtered, and charted externally with absolute mathematical accuracy.

Date / Time	Pressure	Status	Temperature	Total	Ical_test
2026-09-08 13:04:50	27.91	137.56	45.54	138.66	-
2026-09-08 13:04:51	192.60	51.29	2.88	120.49	-
2026-09-08 13:04:52	3.34	13.67	6.27	177.57	-
2026-09-08 13:04:53	40.00	66.26	172.68	121.42	-
2026-09-08 13:04:54	48.77	22.79	9.19	189.90	-
2026-09-08 13:04:55	184.95	61.84	49.21	181.14	-
2026-09-08 13:04:56	167.77	41.06	100.42	149.35	-
2026-09-08 13:04:57	37.50	32.85	122.20	52.60	-
2026-09-08 13:04:58	70.88	121.34	68.44	106.29	-
2026-09-08 13:04:59	191.84	167.56	192.64	171.10	-
2026-09-08 13:05:00	175.57	66.21	69.38	150.60	176.83
2026-09-08 13:05:01	185.24	15.29	114.93	17.73	-
2026-09-08 13:05:02	12.36	188.29	93.98	194.16	-
2026-09-08 13:05:03	44.33	173.43	178.56	81.06	-
2026-09-08 13:05:04	26.81	139.72	29.69	176.85	-
COUNT	25	25	25	25	1
AVERAGE	98.28	98.43	92.00	124.67	176.83
MINIMUM	3.34	13.67	2.88	17.73	176.83
MAXIMUM	192.60	188.29	197.23	194.16	176.83

The complete, updated VBScript code blocks containing the Excel COM automation handles, dynamic cell loop iterations, and structural reporting functions can be reviewed in detail via the provided **[GitHub link](#)**.

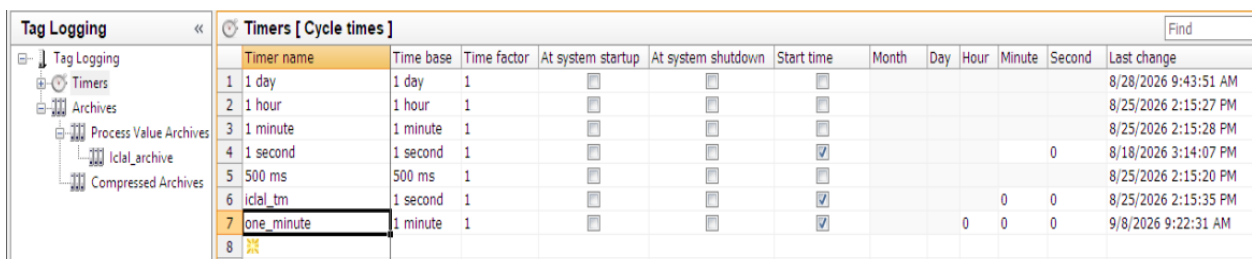
Analyzing Multi-Cycle Archiving

To analyze how SCADA architecture handles variables with varying data dynamics, I introduced a testing scenario based on multi-cycle logging. Industrial systems often monitor high-speed variables (like pressure or current) alongside slow-moving parameters (like ambient temperature or bulk tank levels). Archiving every tag at the same high speed unnecessarily inflates database sizes and impacts system processing performance.



	Process tag	Tag type	Tag name	Archive name	Comment	Acquisition type
1	deneme	Analog	deneme	Iclal_archive		Cyclical, continuous
2	Iclal_test	Analog	Iclal_test	Iclal_archive		Cyclical, continuous
3	new_test	Analog	new_test	Iclal_archive		Cyclical, continuous
4	Pressure	Analog	Pressure	Iclal_archive		Cyclical, continuous
5	Status	Analog	Status	Iclal_archive		Cyclical, continuous
6	Temperature	Analog	Temperature	Iclal_archive		Cyclical, continuous
7	Total	Analog	Total	Iclal_archive		Cyclical, continuous

Within the **Tag Logging** properties under the **Timers [Cycle times]** directory, I created a custom cycle configuration named **iclal_tm** with a time base set to **one minute**. This allows the project to support two completely independent logging frequencies simultaneously.

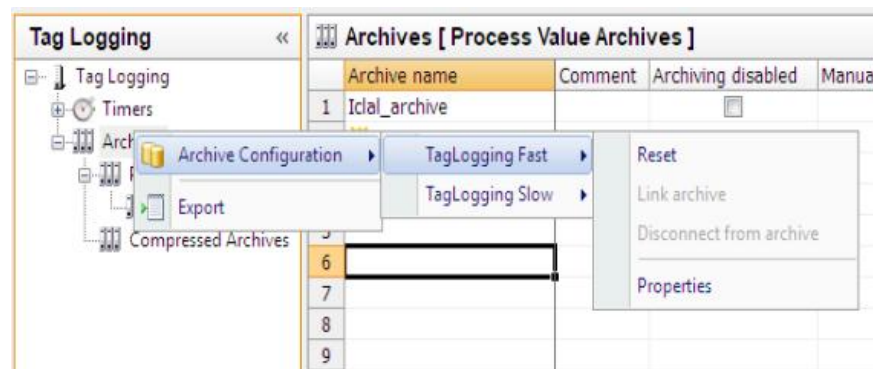


	Timer name	Time base	Time factor	At system startup	At system shutdown	Start time	Month	Day	Hour	Minute	Second	Last change
1	1 day	1 day	1	☐	☐	☐						8/28/2026 9:43:51 AM
2	1 hour	1 hour	1	☐	☐	☐						8/25/2026 2:15:27 PM
3	1 minute	1 minute	1	☐	☐	☐						8/25/2026 2:15:28 PM
4	1 second	1 second	1	☐	☐	☒					0	8/18/2026 3:14:07 PM
5	500 ms	500 ms	1	☐	☐	☐						8/25/2026 2:15:20 PM
6	iclal_tm	1 second	1	☐	☐	☒				0	0	8/25/2026 2:15:35 PM
7	one_minute	1 minute	1	☐	☐	☒			0	0	0	9/8/2026 9:22:31 AM
8												

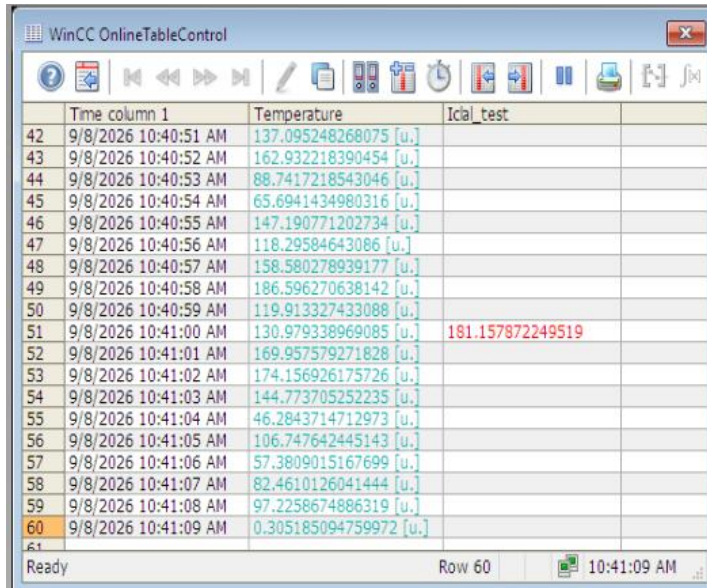
I introduced a new test variable named **Iclal_test** to evaluate this differential logging structure. As shown in the *Archives [Iclal_archive]* directory layout, while the core process variables (Temperature, Pressure, Status, Total) remained assigned to the fast **1 second** acquisition cycle, I specifically modified the properties of **Iclal_test**. I changed its **Acquisition cycle** and **Archiving/display cycle** to the newly defined **iclal_tm (one minute)** interval, allowing it to behave as a slow-moving process point.

Tag Logging		Archives [Iclal_archive]							Find
	Process tag	Archiving disabled	Also in tag	Acquisition cycle	Factor for archiving cycle	Archiving/display cycle	Archiving on	Processing	
1	deneme	☐		500 ms	1	500 ms		Actual value	
2	Iclal_test	☐		icla_tm	1	one_minute		Actual value	
3	new_test	☐		500 ms	1	500 ms		Actual value	
4	Pressure	☐		icla_tm	1	icla_tm		Actual value	
5	Status	☐		icla_tm	1	icla_tm		Actual value	
6	Temperature	☐		icla_tm	1	icla_tm		Actual value	
7	Total	☐		icla_tm	1	icla_tm		Actual value	
8									

During the tag simulation setup, I assigned a **Random** function to Iclal_test to continuously feed it test values. To thoroughly evaluate our database conversion rules and screen responses, I tested it under different simulated conditions. In the **Quality set** dropdown configuration pane, I toggled its status between 0x60 - uncertain - simulated value and 0x80 - good - ok, observing how the runtime engine processes shifting signal statuses across prolonged intervals.



Running highly frequent background simulations across distinct logging cycles can occasionally cause data synchronization conflicts or runtime buffer congestion if parameters are altered mid-operation. To prevent these internal communication faults and stop the runtime memory buffer from overloading, I utilized WinCC's archive maintenance utilities. By expanding the **Tag Logging** tree, right-clicking on **Archive Configuration**, and navigating through both the **TagLogging Fast** and **TagLogging Slow** sub-options, I executed a **Reset** command. Purging the active runtime cache buffer flushes out conflicting temporary histories, resets the transaction queue, and ensures that the system continues to archive shifting **data speeds smoothly without memory leaks or dashboard freezes**.



	Time column 1	Temperature	Iclal_test
42	9/8/2026 10:40:51 AM	137.095248268075 [u.]	
43	9/8/2026 10:40:52 AM	162.932218390454 [u.]	
44	9/8/2026 10:40:53 AM	88.7417218543046 [u.]	
45	9/8/2026 10:40:54 AM	65.6941434980316 [u.]	
46	9/8/2026 10:40:55 AM	147.190771202734 [u.]	
47	9/8/2026 10:40:56 AM	118.29584643086 [u.]	
48	9/8/2026 10:40:57 AM	158.580278939177 [u.]	
49	9/8/2026 10:40:58 AM	186.596270638142 [u.]	
50	9/8/2026 10:40:59 AM	119.913327433088 [u.]	
51	9/8/2026 10:41:00 AM	130.979338969085 [u.]	181.157872249519
52	9/8/2026 10:41:01 AM	169.957579271828 [u.]	
53	9/8/2026 10:41:02 AM	174.156926175726 [u.]	
54	9/8/2026 10:41:03 AM	144.773705252235 [u.]	
55	9/8/2026 10:41:04 AM	46.2843714712973 [u.]	
56	9/8/2026 10:41:05 AM	106.747642445143 [u.]	
57	9/8/2026 10:41:06 AM	57.3809015167699 [u.]	
58	9/8/2026 10:41:07 AM	82.4610126041444 [u.]	
59	9/8/2026 10:41:08 AM	97.2258674886319 [u.]	
60	9/8/2026 10:41:09 AM	0.305185094759972 [u.]	

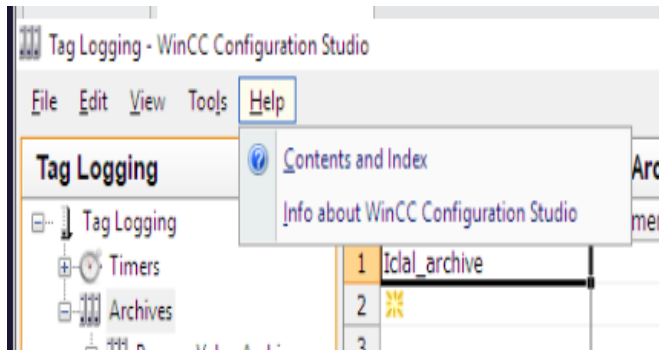
Ready Row 60 10:41:09 AM

When launching the WinCC Runtime screen, the operational impact of this differential configuration becomes instantly clear in the **WinCC OnlineTableControl** log grid. The fast variables update chronologically second-by-second (e.g., 10:40:51, 10:40:52, 10:40:53), generating continuous numeric data rows. In stark contrast, the Iclal_test column remains empty for most of these ticks. It records its random float value strictly once every 60 seconds (at the 10:41:00 minute

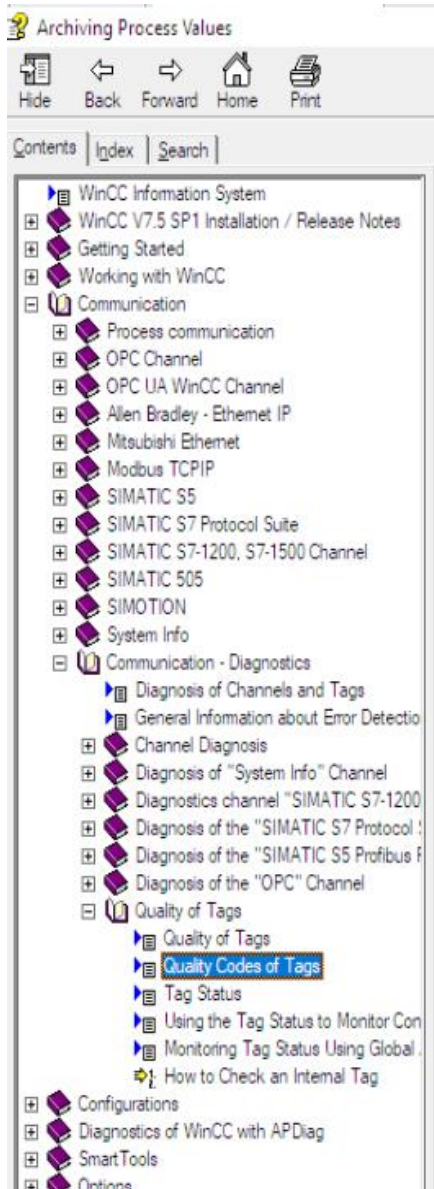
mark), leaving the surrounding rows unlogged until the next full minute passes. This visually proves that the system executes independent logging boundaries perfectly.

Quality codes of Tags

In industrial SCADA environments, monitoring numerical data points alone is not



sufficient. Physical field elements often suffer from power outages, sensor calibration drifts, or instantaneous communication drops. To prevent field operators from making decisions based on stale or corrupted data, it is crucial to monitor the **Quality Code** of each variable.



I opened the **WinCC Configuration Studio Help System** by navigating *Help -> Contents and Index*. Under the *Communication -> Communication - Diagnostics* directory, I audited the **Quality Codes of Tags** specification framework. The documentation states that the first two binary digits define the structural status: 00 for Bad, 01 for Uncertain, and 10 or 11 for Good. For instance, a hex code of **0x60** signals an *Uncertain - Simulated value*, whereas **0x80** confirms a perfectly healthy *Good - OK* data link.

Realizing the immense value of quality codes for industrial traceability, I updated both the SQL Server database schema and the background **VBScript** structure. The updated script utilizes the quality property of each HMI tag object in WinCC Runtime. It extracts these evaluation statuses in real-time alongside the raw data fields, pushing both datasets into our unified tables automatically.

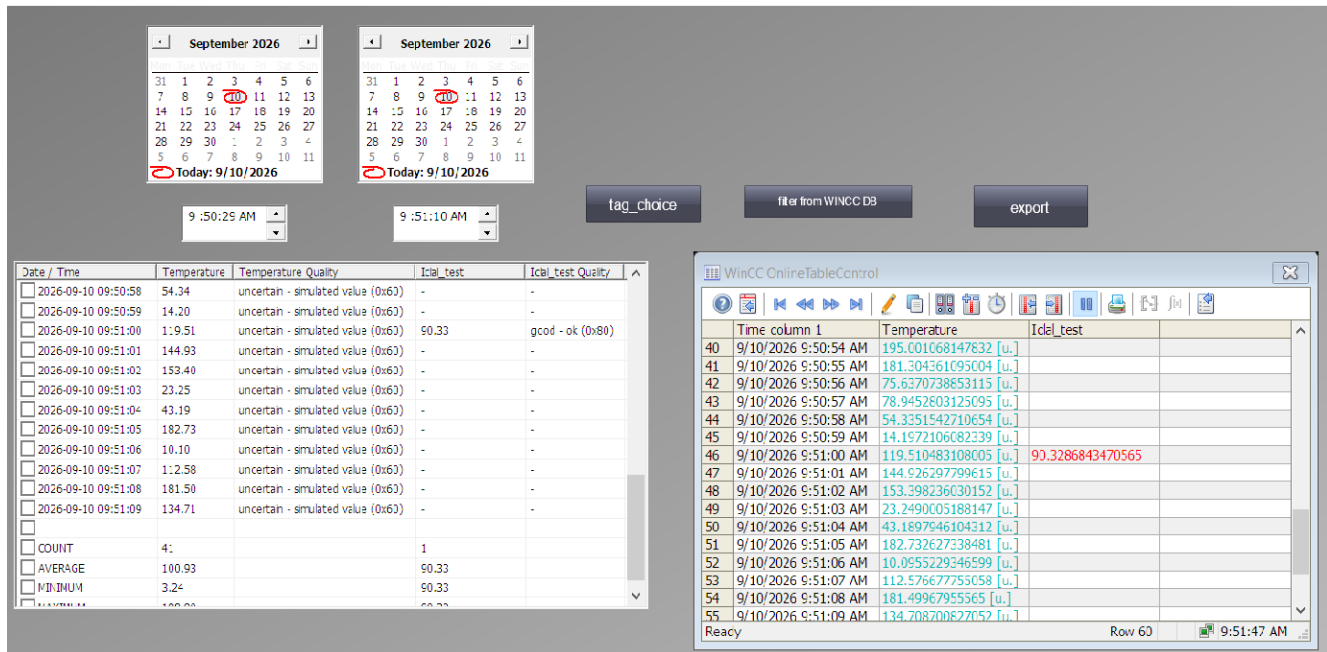
Quality

The first two digits specify the quality of the tag.

	Q	Q	S	S	S	S	L	L
	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bad – The value is not useful	0	0	-	-	-	-	-	-
Uncertain – The quality of the value is less than normal, but the value may still be useful.	0	1	-	-	-	-	-	-
Good (non-cascade) – The quality of the value is good. Possible alarm conditions may be indicated by the sub-status.	1	0	-	-	-	-	-	-
Good (cascade) – The value may be used in control.	1	1	-	-	-	-	-	-

0x2B	Bad	Process related – No maintenance	0	0	1	0	1	0	1	1
0x68	Uncertain	Maintenance demanded	0	1	1	0	1	0	-	-
0x60	Uncertain	Simulated value – Set when the process value is written by the operator while the block is in manual mode.	0	1	1	0	0	0	-	-
0x64	Uncertain	Sensor calibration	0	1	1	0	0	1	-	-
0x5C	Uncertain	Configuration error	0	1	0	1	1	1	-	-
0x58	Uncertain	Subnormal	0	1	0	1	1	0	-	-

0x88	Good (non-cascade)	Active advisory alarm – Set if the value is good and the block has an active alarm with a priority less than 8.	1	0	0	0	1	0	-	-
0xA8	Good (non-cascade)	Maintenance demanded	1	0	1	0	1	0	-	-
0xA4	Good (non-cascade)	Maintenance required	1	0	1	0	0	1	-	-
0x8C	Good (non-cascade)	Function check – Local override	1	0	1	1	1	1	-	-
0x80	Good (non-cascade)	OK – No error or special condition is associated with this value.	1	0	0	0	0	0	-	-

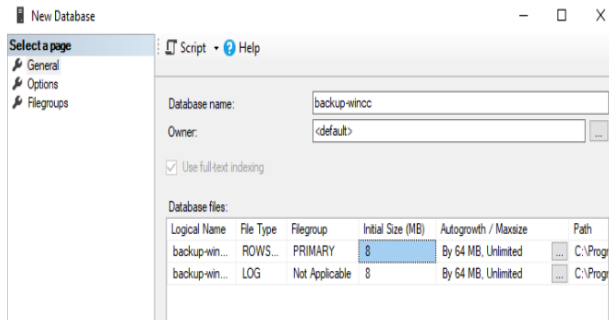


During operational testing, the advanced table dashboard beautifully reflects the stability of our pipeline infrastructure. Although the WinCC documentation and system configuration display these quality codes in **hexadecimal format** (such as 0x60 or 0x80), the database automatically stores and processes these incoming values as **decimal integers** (96 and 128, respectively). To resolve this format discrepancy and ensure the logs remain human-readable for operators, I implemented conditional **If-Else statements** inside my VBScript. The logic evaluates the incoming decimal integer, maps it to its corresponding hex status definition, and explicitly writes out the clear textual representation into the quality columns.

The updated VBScript code, including these advanced conditional loops and translation structures, can be found directly within the provided **[GitHub link](#)**. Export button will be the same as it is.

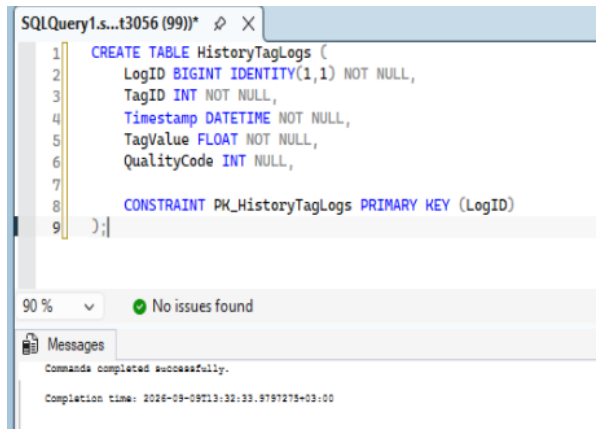
Database Performance Optimization / Back-up

To complete the data management framework of the SCADA system, I configured a dedicated backup database and designed optimized table structures in SQL Server Management Studio (SSMS) to ensure high-performance data logging as the project scales.

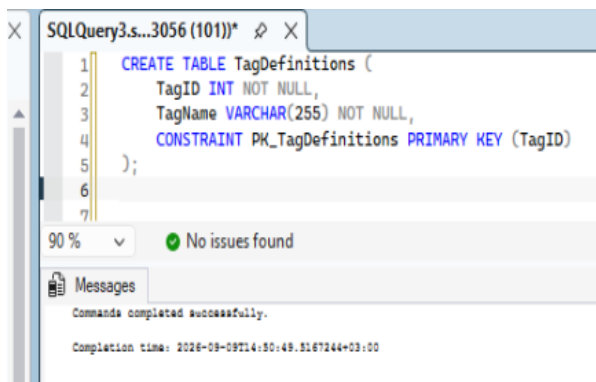


As shown in the "New Database" properties panel, I established a dedicated backup environment named **backup-wincc**. To ensure data integrity, I set the initial sizes for the primary database data file (backup-wincc) and the transaction log file (backup-wincc_LOG) to **8 Megabytes** and **64 Megabytes**, respectively. Both files are

configured with **Autogrowth enabled by 64 Megabytes, Unrestricted**, allowing the system storage to dynamically scale as historical data flows in from WinCC.



To log incoming historical process values efficiently, I created an optimized transactional data table named HistoryTagLogs. The structural table definition was executed using the following SQL query script.



In this design, LogID serves as the primary key using a BIGINT format with an identity increment (IDENTITY (1,1)) to handle billions of unique row logs. During the system design phase, I evaluated index and constraint behaviors for large-scale operations.

When tracking vast industrial setups where tag counts expand into the tens of thousands, maintaining a strict **Foreign Key** constraint linked to static definition tables can cause write-speed bottlenecks. To prevent writing

delays, dropping (DROP) or avoiding heavy foreign key relationships on this rapid-fire transactional table will immediately improve data ingestion speeds by **3 to 4 times**, ensuring the database can keep up with millisecond-level WinCC logs.

To store the static descriptions and names of each process tag without duplicating text strings in the heavy log table, I built a relational index table named TagDefinitions. The table layout was deployed using the following SQL architecture script:

ENG\WINCC....toryTagLogs			
Column Name	Data Type	Allow Nulls	
LogID	bigint	☐	
TagID	int	☐	
Timestamp	datetime	☐	
TagValue	float	☐	
QualityCode	int	☒	
		☐	

ENG\WINCC.b...Definitions			
Column Name	Data Type	Allow Nulls	
TagID	int	☐	
TagName	varchar(255)	☐	
		☐	

To streamline data maintenance and automate the background backup routine within the SQL Server architecture, I developed a custom Stored Procedure. Relying on manual exports or rigid WinCC background limits can be prone to errors; creating a dedicated procedure inside the SQL Server instance allows the database engine to handle complex extraction logic independently and efficiently.

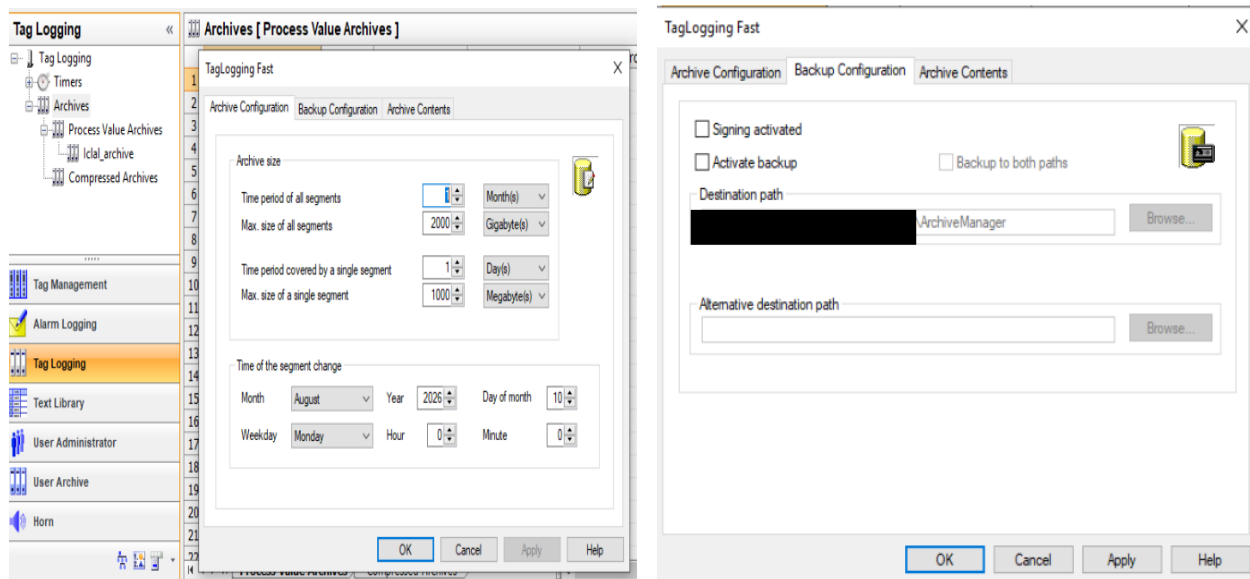
```

SQLQuery16.sq...rt3056 (96))  SQLQuery15.s...rt3056 (99))*  SQLQuery14.s...rt3056 (94))*  S
1  USE [backup-wincc]
2  GO
3
4  IF OBJECT_ID('dbo.Sp_WinCC_AllTags_Backup', 'P') IS NOT NULL
5      DROP PROCEDURE dbo.Sp_WinCC_AllTags_Backup;
6  GO
7
8  CREATE PROCEDURE dbo.Sp_WinCC_AllTags_Backup
9      @StartTime VARCHAR(30),
10     @EndTime VARCHAR(30)
11  AS
12  BEGIN
13      SET NOCOUNT ON;
14
15      DECLARE @CurrentTagID INT;
16      DECLARE @DynamicSQL NVARCHAR(MAX);
17
18      DECLARE Tag_Cursor CURSOR FOR
19      SELECT TagID FROM [backup-wincc].[dbo].[TagDefinitions] ORDER BY TagID;
20
21      OPEN Tag_Cursor;
22      FETCH NEXT FROM Tag_Cursor INTO @CurrentTagID;
23
24      WHILE @@FETCH_STATUS = 0
25      BEGIN
26          --
27      END
28  END
29
30  Messages
31
32  Completion time: 2026-09-14T11:15:18.7878666+03:00

```

SQL script can be found directly within the provided ***GitHub link***.

I opened a new query editor in SSMS under the backup-wincc target database to code the automation routine. The script starts by checking if the procedure already exists (IF OBJECT_ID... IS NOT NULL DROP PROCEDURE...) to allow clean overrides during structural updates. I then initialized a stored procedure named `dbo.Sp_WinCC_AllTags_Backup` which accepts two evaluation variables: `StartTime` and `EndTime`. Inside the procedure, I declared a server-side **Cursor** (`DECLARE Tag_Cursor CURSOR FOR...`) to query the `TagID` indices directly from the static metadata catalog (`[backup-wincc].[dbo].[TagDefinitions]`). This structure enables a systematic loop (`WHILE FETCH_STATUS = 0`) that reads the historical dataset tag-by-tag, building and running dynamic SQL injection strings (`NVARCHAR(MAX)`) completely on the fly based on the time windows provided by the operator.

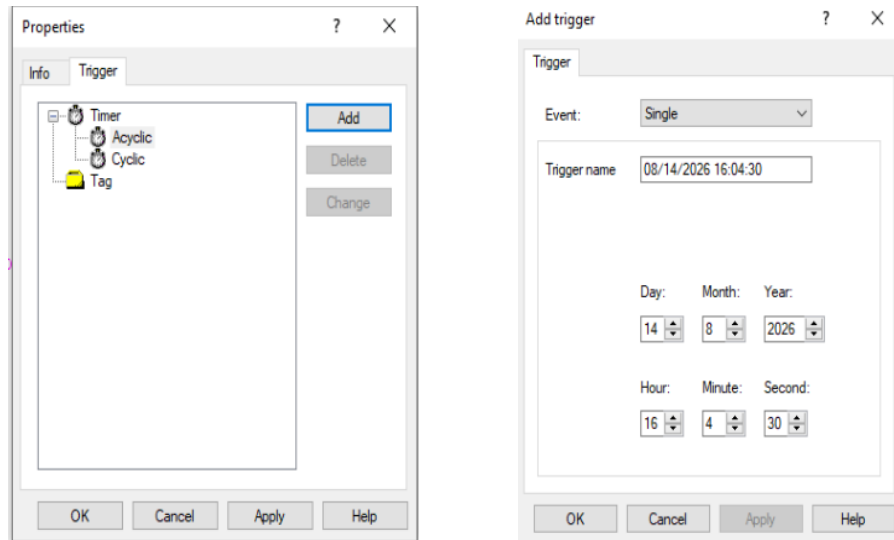


To align this database-level automation with the SCADA ecosystem, I cross-checked the archive constraints inside the WinCC **Tag Logging Fast** configuration window. As shown in the setup dashboards, the local runtime cache is restricted to a tight schedule—allocating a maximum size of **2000 Megabytes** per segmentation loop before it flushes old registers. By deploying the database-side stored procedure, I created an independent, high-performance archival mirror. This guarantees that even when the local WinCC fast cache resets its segments to save disk space, the data remains safely indexed and restorable inside the core server layer.

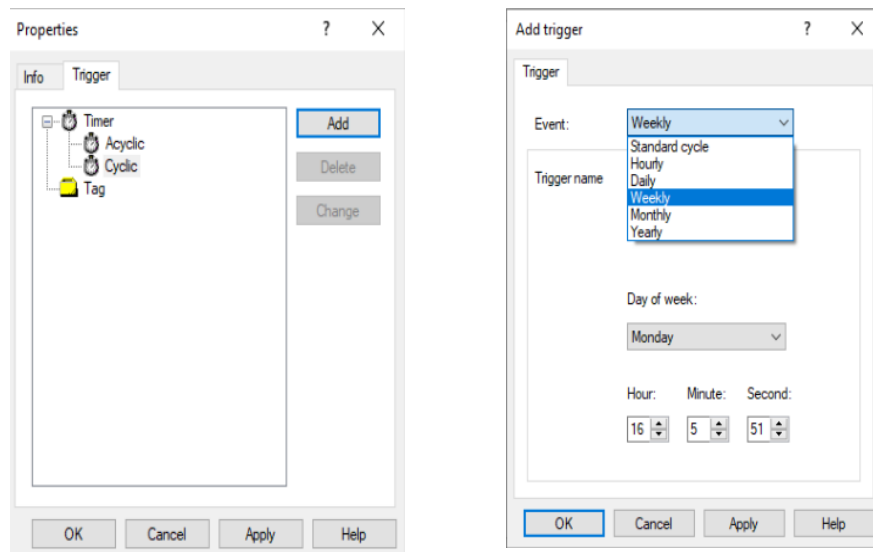
The updated VBScript code of `TagDefinitions` can be found directly within the provided **[GitHub link](#)**. and for **[TimeTrigger for backup](#)** of `HistoryTagLogs` table.



To ensure that the developed database integration routines and stored procedures run automatically without human intervention, I configured the execution triggers within the WinCC Global Script architecture.

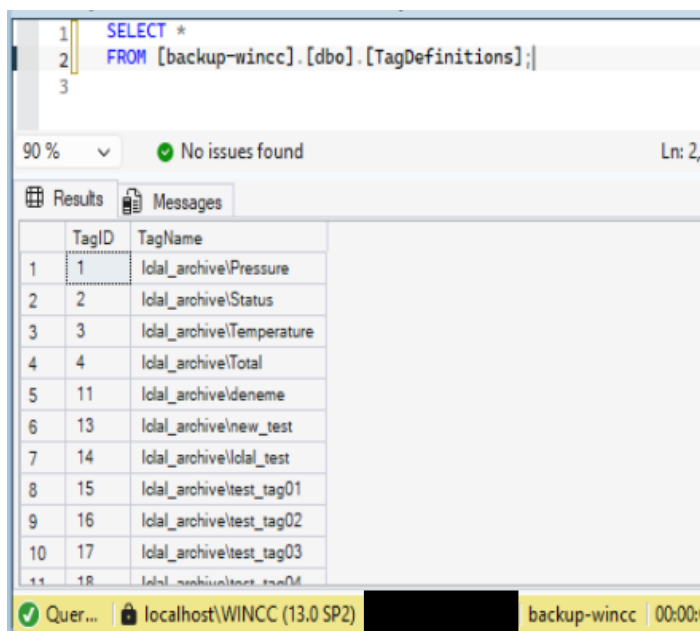


To verify the script's stability during the initial design phase, I opened the script properties and added an **Acyclic** timer trigger. As shown in the configuration dialog, this allows assigning a specific, one-time execution window locked to an exact date and time (e.g., 14/09/2026 16:04:30). This setup is ideal for debugging and checking database connection strings on demand before deploying them into production.



While the acyclic trigger was used for manual testing, a continuous, automated loop is required to handle routine tasks like weekly data backups or monthly report generations. To achieve this, the execution mechanism can be easily switched to a **Cyclic** trigger structure.

Within the *Add trigger* configuration panel, opening the **Event** dropdown menu reveals multiple recurring standard cycles: **Daily**, **Weekly**, **Monthly**, and **Yearly**. For a standard factory setup, configuring a **Weekly** cycle assigned to a specific day (such as *Monday* at 16:10:51) ensures that the database maintenance scripts run smoothly during low-traffic shifts, preventing any potential runtime processor congestion.



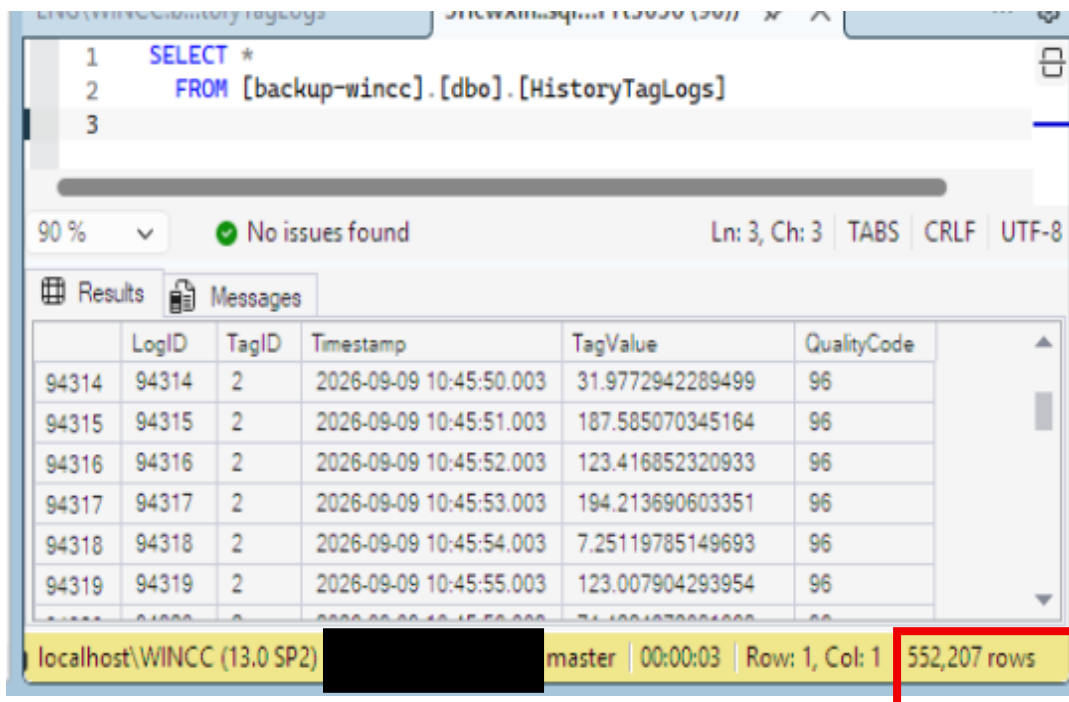
	TagID	TagName
1	1	lclal_archive\Pressure
2	2	lclal_archive\Status
3	3	lclal_archive\Temperature
4	4	lclal_archive\Total
5	11	lclal_archive\deneme
6	13	lclal_archive\new_test
7	14	lclal_archive\lclal_test
8	15	lclal_archive\test_tag01
9	16	lclal_archive\test_tag02
10	17	lclal_archive\test_tag03
11	18	lclal_archive\test_tag04

After binding the global VBScript to the acyclic trigger window, the script successfully executed on schedule, transferring the active data from the WinCC runtime system straight into the dedicated backup-wincc database instance. To verify that the execution was completed flawlessly, I performed structural data queries inside SQL Server Management Studio (SSMS).

As illustrated in the first results pane, the system successfully populated the static rows with individual **TagID** mappings matched against their exact SCADA logging pathways (e.g., TagID 1 for lclal_archive\Pressure, TagID 2 for lclal_archive\Status, etc.). This confirms that our descriptive schema tracks the tags perfectly without bloating the main transactional tables.

The second runtime results window confirm the ultimate success of the database deployment. The table features an immense collection of logs (reaching over **552,207 rows** seamlessly). Each row captures a unique LogID, links to the static metadata via TagID, stamps the exact millisecond-level Timestamp, records the high-precision float TagValue, and includes the raw decimal QualityCode (such as **96 in decimal**, which corresponds to our simulated 0x60 status code in hexadecimal). (*Database logs in decimal.*)

This verified dataset completely validates that our automated loop bridges the gap between active SCADA animations and heavy-duty, industrial-grade SQL storage arrays effortlessly.



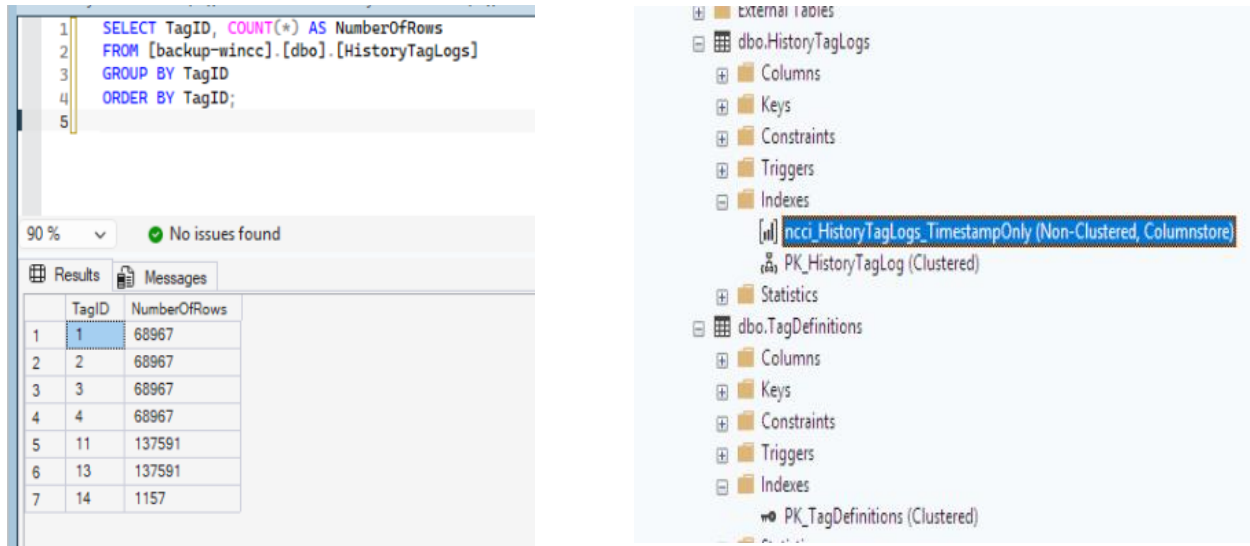
The screenshot shows a SQL query window with the following SQL code:

```
1 SELECT *
2 FROM [backup-wincc].[dbo].[HistoryTagLogs]
3
```

The results window displays a table with the following columns: LogID, TagID, Timestamp, TagValue, and QualityCode. The table contains 552,207 rows. The status bar at the bottom indicates '552,207 rows'.

LogID	TagID	Timestamp	TagValue	QualityCode
94314	94314	2026-09-09 10:45:50.003	31.9772942289499	96
94315	94315	2026-09-09 10:45:51.003	187.585070345164	96
94316	94316	2026-09-09 10:45:52.003	123.416852320933	96
94317	94317	2026-09-09 10:45:53.003	194.213690603351	96
94318	94318	2026-09-09 10:45:54.003	7.25119785149693	96
94319	94319	2026-09-09 10:45:55.003	123.007904293954	96

localhost\WINCC (13.0 SP2) master | 00:00:03 | Row: 1, Col: 1 | 552,207 rows



The screenshot displays a SQL Server Enterprise Manager interface. On the left, a query window shows a SQL query: `SELECT TagID, COUNT(*) AS NumberOfRows FROM [backup-wincc].[dbo].[HistoryTagLogs] GROUP BY TagID ORDER BY TagID;`. Below the query, the 'Results' tab shows a table with two columns: 'TagID' and 'NumberOfRows'. The table contains seven rows of data. On the right, the 'Object Explorer' pane shows the database structure, with the 'Indexes' folder expanded for the 'dbo.HistoryTagLogs' table. A new index, 'ncci_HistoryTagLogs_TimestampOnly (Non-Clustered, Columnstore)', is highlighted.

TagID	NumberOfRows
1	68967
2	68967
3	68967
4	68967
11	137591
13	137591
14	1157

I executed a structured grouping query to count the total rows generated per tag ID within the backup ecosystem. As displayed in the first query results window, the system accurately displays the data density distribution across different variables. For instance, the core parameters (TagID 1, 2, 3, and 4) each contain exactly **68,967 rows**, while other archived parameters have higher densities reaching up to **137,591 rows**.

Looking at these metrics, I realized that as the SCADA system continues to log hundreds of thousands of entries daily, the transaction history will quickly scale into tens of millions of rows. With a rapidly growing dataset, performing historical time-range filtering, trend charts, or automated reports would experience significant query execution delays and server load overhead.

When the dbo.HistoryTagLogs table was initially built, the system automatically generated a traditional **Clustered Index** keyed to the primary identification field (PK_HistoryTagLogs). While a clustered index is excellent for basic row updates, it organizes data sequentially row-by-row, which is highly inefficient for handling mass historical data aggregations or filtering specific columns over vast date ranges.

To eliminate this query bottleneck and optimize historical reporting performance, I integrated an enterprise-level indexing strategy. I created a **Non-Clustered Columnstore Index** specifically targeted at the time-series logs, visible in the object tree layout as ncci_HistoryTagLogs_TimestampOnly (Non-Clustered, Columnstore). Columnstore technology completely changes how the database engine reads data—instead of parsing entire horizontal rows, it compresses and searches columns vertically. This mechanism allows the database engine to isolate and scan timestamp rows instantaneously without reading unnecessary sensor parameters.

```

1 SET STATISTICS IO ON;
2 SET STATISTICS TIME ON;
3 GO
4 SELECT * FROM [backup-wincc].[dbo].[HistoryTagLogs]
5 WHERE Timestamp >= '2026-09-08 00:00:00'
6 AND Timestamp <= '2026-09-08 23:59:59'
7 AND TagID = 1;
8
9

```

150 % No issues found

Results Messages

SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.
SQL Server parse and compile time:
CPU time = 0 ms, elapsed time = 0 ms.

(17595 rows affected)
Table 'HistoryTagLogs'. Scan count 1, logical reads 2812, physical reads 0, read-ahead

SQL Server Execution Times:
CPU time = 47 ms, elapsed time = 210 ms.

Completion time: 2026-09-16T13:57:35.7682050+03:00

```

1 SET STATISTICS IO ON;
2 SET STATISTICS TIME ON;
3 GO
4 SELECT * FROM [backup-wincc].[dbo].[HistoryTagLogs]
5 WHERE Timestamp >= '2026-09-08 00:00:00'
6 AND Timestamp <= '2026-09-08 23:59:59'
7 AND TagID = 1;

```

0 % No issues found

Results Messages

(17595 rows affected)
Table 'HistoryTagLogs'. Scan count 1, logical reads 2812, physical reads 0, read-ahead

SQL Server Execution Times:
CPU time = 32 ms, elapsed time = 127 ms.

Completion time: 2026-09-16T15:19:22.5952724+03:00

To quantify the speed improvements, I ran benchmark test queries filtering specific time boundaries across the **552,207 rows** database partition before and after applying the structural change:

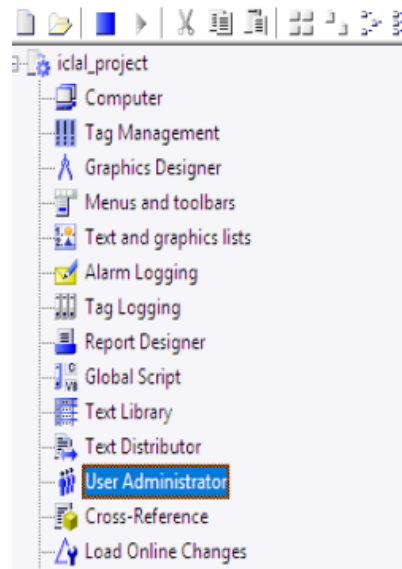
Before Optimization: The analytical search query took **210 milliseconds** to execute using standard clustered row-scanning.

After Optimization: The query execution speed dropped significantly to just **127 milliseconds**.

This measurable latency reduction proves that integrating a columnstore architecture provides highly efficient read speeds, guaranteeing that operator analytics screens remain lightning-fast even when querying years of manufacturing history.

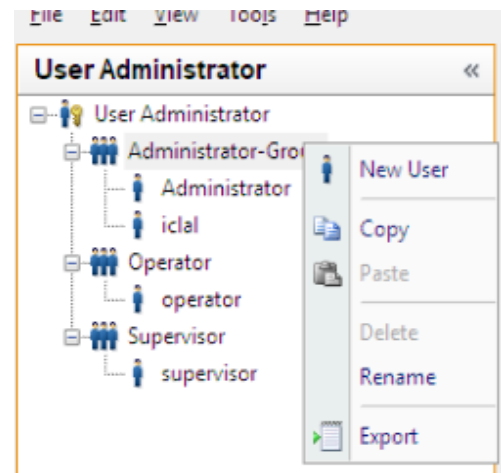
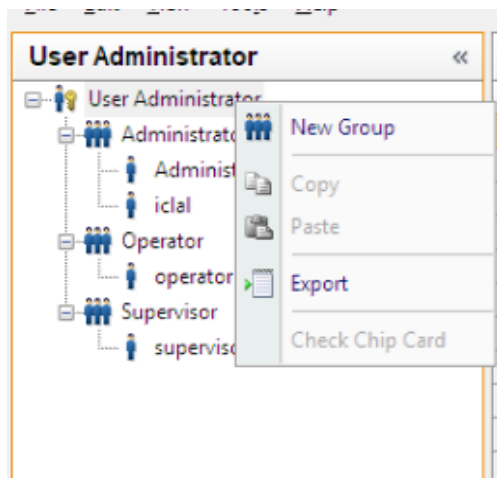
SQL script of index can be found directly within the provided **GitHub link**.

Role-Based User Management



To protect the industrial control environment from unauthorized operational changes and restrict critical system access, I configured a comprehensive security layer using the WinCC **User Administrator** platform. In field environments, enforcing role-based privileges prevents non-technical operators from accidentally modifying control loop scripts or clearing historical tag logs.

I navigated to the main WinCC Explorer application layout and scrolled through the tree nodes. I double-clicked on **User Administrator** to initialize the structural security administration panel.



Inside the configuration dashboard, I right clicked on the root workspace to define structural organizational profiles using **New Group** and **New User** actions (**Figure 1**). I organized the hierarchy into standard industrial operation layers: Administrator-Group, Operator, Supervisor, and a dedicated testing group named **icla_test**. Under the test group, I mapped an individual login profile named **test**.

The screenshot shows the 'User Administrator' window. On the left is a tree view of the user hierarchy: User Administrator (Group), Administrator (User), iclal (User), Operator (Group), operator (User), Supervisor (Group), supervisor (User), iclal_test (Group), and test (User). On the right is a table titled 'Users [iclal_test]' with columns: User name, Group name, Password, and Logon. The table contains 10 rows, with the first row having 'test' as the user name and 'ical_test' as the group name. The Password column for the first row is highlighted in red.

	User name	Group name	Password	Logon
1	test	ical_test		
2				
3				
4				
5				
6				
7				
8				
9				
10				

To secure the newly created profiles, I navigated the user properties window. Under the **Selection** and **General** grids, I targeted the password parameter and opened the **Change Password** pop-up utility (**Figure 2**). Here, I locked down the profile with a unique, encrypted password to verify personnel credentials during runtime login challenges.

The screenshot shows the 'Properties - User' window. It has several tabs: Selection, General, Login, Logout, and Web. The 'General' tab is active, showing fields for User name (test), Group name (ical_test), and Password (highlighted in red). The 'Change Password' dialog box is open over the 'General' tab, with fields for New password, Verify password, and Security. The dialog box has 'OK' and 'Cancel' buttons.

To implement a strict **Least Privilege Architecture**, I audited and modified the functional checklist under the **Authorizations** console for each group profile:

The Administrator group and the explicit user account iclal were granted complete global control over the system. Their matrices have every single operational checkbox marked (**Enable**), including sensitive permissions like *User Administration* (1), *Archive Editing* (15), *Action Editing* (16), and *Project Manager* (17).

User Administrator	Authorizations [Administrator]																																												
User Administrator Administrator-Group Administrator iclal Operator operator Supervisor supervisor	<table> <tr> <th>Function</th><th>Enable</th></tr> <tr><td>1 User Administration</td><td>☒</td></tr> <tr><td>2 Value input</td><td>☒</td></tr> <tr><td>3 Process controlling</td><td>☒</td></tr> <tr><td>4 Picture Editing</td><td>☒</td></tr> <tr><td>5 Change picture</td><td>☒</td></tr> <tr><td>6 Window selection</td><td>☒</td></tr> <tr><td>7 Hardcopy</td><td>☒</td></tr> <tr><td>8 Confirm alarms</td><td>☒</td></tr> <tr><td>9 Lock alarms</td><td>☒</td></tr> <tr><td>10 Free alarms</td><td>☒</td></tr> <tr><td>11 Message Editing</td><td>☒</td></tr> <tr><td>12 Start archive</td><td>☒</td></tr> <tr><td>13 Stop archive</td><td>☒</td></tr> <tr><td>14 Edit archive values</td><td>☒</td></tr> <tr><td>15 Archive Editing</td><td>☒</td></tr> <tr><td>16 Action Editing</td><td>☒</td></tr> <tr><td>17 Project Manager</td><td>☒</td></tr> <tr><td>18 Activate remote</td><td>☒</td></tr> <tr><td>19 Configure remote</td><td>☒</td></tr> <tr><td>20 Web Access - monitoring only</td><td>☒</td></tr> <tr><td>21</td><td></td></tr> </table>	Function	Enable	1 User Administration	☒	2 Value input	☒	3 Process controlling	☒	4 Picture Editing	☒	5 Change picture	☒	6 Window selection	☒	7 Hardcopy	☒	8 Confirm alarms	☒	9 Lock alarms	☒	10 Free alarms	☒	11 Message Editing	☒	12 Start archive	☒	13 Stop archive	☒	14 Edit archive values	☒	15 Archive Editing	☒	16 Action Editing	☒	17 Project Manager	☒	18 Activate remote	☒	19 Configure remote	☒	20 Web Access - monitoring only	☒	21	
Function	Enable																																												
1 User Administration	☒																																												
2 Value input	☒																																												
3 Process controlling	☒																																												
4 Picture Editing	☒																																												
5 Change picture	☒																																												
6 Window selection	☒																																												
7 Hardcopy	☒																																												
8 Confirm alarms	☒																																												
9 Lock alarms	☒																																												
10 Free alarms	☒																																												
11 Message Editing	☒																																												
12 Start archive	☒																																												
13 Stop archive	☒																																												
14 Edit archive values	☒																																												
15 Archive Editing	☒																																												
16 Action Editing	☒																																												
17 Project Manager	☒																																												
18 Activate remote	☒																																												
19 Configure remote	☒																																												
20 Web Access - monitoring only	☒																																												
21																																													

User Administrator	Authorizations [iclal]																																												
User Administrator Administrator-Group Administrator iclal Operator operator Supervisor supervisor	<table> <tr> <th>Function</th><th>Enable</th></tr> <tr><td>1 User Administration</td><td>☒</td></tr> <tr><td>2 Value input</td><td>☒</td></tr> <tr><td>3 Process controlling</td><td>☒</td></tr> <tr><td>4 Picture Editing</td><td>☒</td></tr> <tr><td>5 Change picture</td><td>☒</td></tr> <tr><td>6 Window selection</td><td>☒</td></tr> <tr><td>7 Hardcopy</td><td>☒</td></tr> <tr><td>8 Confirm alarms</td><td>☒</td></tr> <tr><td>9 Lock alarms</td><td>☒</td></tr> <tr><td>10 Free alarms</td><td>☒</td></tr> <tr><td>11 Message Editing</td><td>☒</td></tr> <tr><td>12 Start archive</td><td>☒</td></tr> <tr><td>13 Stop archive</td><td>☒</td></tr> <tr><td>14 Edit archive values</td><td>☒</td></tr> <tr><td>15 Archive Editing</td><td>☒</td></tr> <tr><td>16 Action Editing</td><td>☒</td></tr> <tr><td>17 Project Manager</td><td>☒</td></tr> <tr><td>18 Activate remote</td><td>☒</td></tr> <tr><td>19 Configure remote</td><td>☒</td></tr> <tr><td>20 Web Access - monitoring only</td><td>☒</td></tr> <tr><td>21</td><td></td></tr> </table>	Function	Enable	1 User Administration	☒	2 Value input	☒	3 Process controlling	☒	4 Picture Editing	☒	5 Change picture	☒	6 Window selection	☒	7 Hardcopy	☒	8 Confirm alarms	☒	9 Lock alarms	☒	10 Free alarms	☒	11 Message Editing	☒	12 Start archive	☒	13 Stop archive	☒	14 Edit archive values	☒	15 Archive Editing	☒	16 Action Editing	☒	17 Project Manager	☒	18 Activate remote	☒	19 Configure remote	☒	20 Web Access - monitoring only	☒	21	
Function	Enable																																												
1 User Administration	☒																																												
2 Value input	☒																																												
3 Process controlling	☒																																												
4 Picture Editing	☒																																												
5 Change picture	☒																																												
6 Window selection	☒																																												
7 Hardcopy	☒																																												
8 Confirm alarms	☒																																												
9 Lock alarms	☒																																												
10 Free alarms	☒																																												
11 Message Editing	☒																																												
12 Start archive	☒																																												
13 Stop archive	☒																																												
14 Edit archive values	☒																																												
15 Archive Editing	☒																																												
16 Action Editing	☒																																												
17 Project Manager	☒																																												
18 Activate remote	☒																																												
19 Configure remote	☒																																												
20 Web Access - monitoring only	☒																																												
21																																													

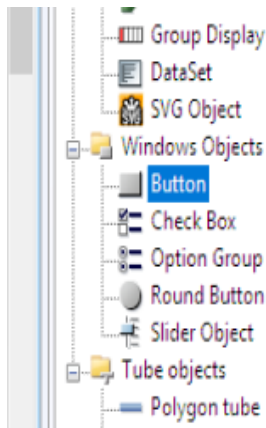
The operator profile was assigned highly restrictive execution access. To prevent unauthorized system parameter configurations, checkboxes for core back-end management capabilities—such as *User Administration*, *Picture Editing*, *Archive Editing*, and *Project Manager*—were kept blank. Operators are strictly allowed to perform basic real-time adjustments like *Value Input* and *Process Controlling*.

User Administrator	Authorizations [operator]																																												
User Administrator Administrator-Group Administrator iclal Operator operator Supervisor supervisor	<table> <tr> <th>Function</th><th>Enable</th></tr> <tr><td>1 User Administration</td><td>☐</td></tr> <tr><td>2 Value input</td><td>☐</td></tr> <tr><td>3 Process controlling</td><td>☐</td></tr> <tr><td>4 Picture Editing</td><td>☐</td></tr> <tr><td>5 Change picture</td><td>☐</td></tr> <tr><td>6 Window selection</td><td>☐</td></tr> <tr><td>7 Hardcopy</td><td>☐</td></tr> <tr><td>8 Confirm alarms</td><td>☐</td></tr> <tr><td>9 Lock alarms</td><td>☐</td></tr> <tr><td>10 Free alarms</td><td>☐</td></tr> <tr><td>11 Message Editing</td><td>☐</td></tr> <tr><td>12 Start archive</td><td>☐</td></tr> <tr><td>13 Stop archive</td><td>☐</td></tr> <tr><td>14 Edit archive values</td><td>☐</td></tr> <tr><td>15 Archive Editing</td><td>☐</td></tr> <tr><td>16 Action Editing</td><td>☐</td></tr> <tr><td>17 Project Manager</td><td>☐</td></tr> <tr><td>18 Activate remote</td><td>☐</td></tr> <tr><td>19 Configure remote</td><td>☐</td></tr> <tr><td>20 Web Access - monitoring only</td><td>☒</td></tr> <tr><td>21</td><td></td></tr> </table>	Function	Enable	1 User Administration	☐	2 Value input	☐	3 Process controlling	☐	4 Picture Editing	☐	5 Change picture	☐	6 Window selection	☐	7 Hardcopy	☐	8 Confirm alarms	☐	9 Lock alarms	☐	10 Free alarms	☐	11 Message Editing	☐	12 Start archive	☐	13 Stop archive	☐	14 Edit archive values	☐	15 Archive Editing	☐	16 Action Editing	☐	17 Project Manager	☐	18 Activate remote	☐	19 Configure remote	☐	20 Web Access - monitoring only	☒	21	
Function	Enable																																												
1 User Administration	☐																																												
2 Value input	☐																																												
3 Process controlling	☐																																												
4 Picture Editing	☐																																												
5 Change picture	☐																																												
6 Window selection	☐																																												
7 Hardcopy	☐																																												
8 Confirm alarms	☐																																												
9 Lock alarms	☐																																												
10 Free alarms	☐																																												
11 Message Editing	☐																																												
12 Start archive	☐																																												
13 Stop archive	☐																																												
14 Edit archive values	☐																																												
15 Archive Editing	☐																																												
16 Action Editing	☐																																												
17 Project Manager	☐																																												
18 Activate remote	☐																																												
19 Configure remote	☐																																												
20 Web Access - monitoring only	☒																																												
21																																													

User Administrator	Authorizations [supervisor]																																												
User Administrator Administrator-Group Administrator iclal Operator operator Supervisor supervisor	<table> <tr> <th>Function</th><th>Enable</th></tr> <tr><td>1 User Administration</td><td>☐</td></tr> <tr><td>2 Value input</td><td>☒</td></tr> <tr><td>3 Process controlling</td><td>☐</td></tr> <tr><td>4 Picture Editing</td><td>☒</td></tr> <tr><td>5 Change picture</td><td>☐</td></tr> <tr><td>6 Window selection</td><td>☐</td></tr> <tr><td>7 Hardcopy</td><td>☐</td></tr> <tr><td>8 Confirm alarms</td><td>☐</td></tr> <tr><td>9 Lock alarms</td><td>☐</td></tr> <tr><td>10 Free alarms</td><td>☐</td></tr> <tr><td>11 Message Editing</td><td>☐</td></tr> <tr><td>12 Start archive</td><td>☐</td></tr> <tr><td>13 Stop archive</td><td>☐</td></tr> <tr><td>14 Edit archive values</td><td>☐</td></tr> <tr><td>15 Archive Editing</td><td>☐</td></tr> <tr><td>16 Action Editing</td><td>☐</td></tr> <tr><td>17 Project Manager</td><td>☐</td></tr> <tr><td>18 Activate remote</td><td>☐</td></tr> <tr><td>19 Configure remote</td><td>☐</td></tr> <tr><td>20 Web Access - monitoring only</td><td>☐</td></tr> <tr><td>21</td><td></td></tr> </table>	Function	Enable	1 User Administration	☐	2 Value input	☒	3 Process controlling	☐	4 Picture Editing	☒	5 Change picture	☐	6 Window selection	☐	7 Hardcopy	☐	8 Confirm alarms	☐	9 Lock alarms	☐	10 Free alarms	☐	11 Message Editing	☐	12 Start archive	☐	13 Stop archive	☐	14 Edit archive values	☐	15 Archive Editing	☐	16 Action Editing	☐	17 Project Manager	☐	18 Activate remote	☐	19 Configure remote	☐	20 Web Access - monitoring only	☐	21	
Function	Enable																																												
1 User Administration	☐																																												
2 Value input	☒																																												
3 Process controlling	☐																																												
4 Picture Editing	☒																																												
5 Change picture	☐																																												
6 Window selection	☐																																												
7 Hardcopy	☐																																												
8 Confirm alarms	☐																																												
9 Lock alarms	☐																																												
10 Free alarms	☐																																												
11 Message Editing	☐																																												
12 Start archive	☐																																												
13 Stop archive	☐																																												
14 Edit archive values	☐																																												
15 Archive Editing	☐																																												
16 Action Editing	☐																																												
17 Project Manager	☐																																												
18 Activate remote	☐																																												
19 Configure remote	☐																																												
20 Web Access - monitoring only	☐																																												
21																																													

The supervisor profile was configured as an intermediate layer between administrators and floor staff . Supervisors were granted permissions to perform system-level overrides, such as managing screens or acknowledging critical data alerts, but they lack root-level privileges to modify user account configurations or backend database tables.

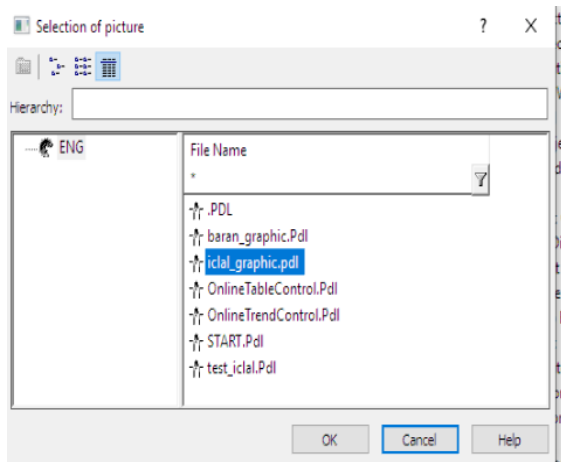
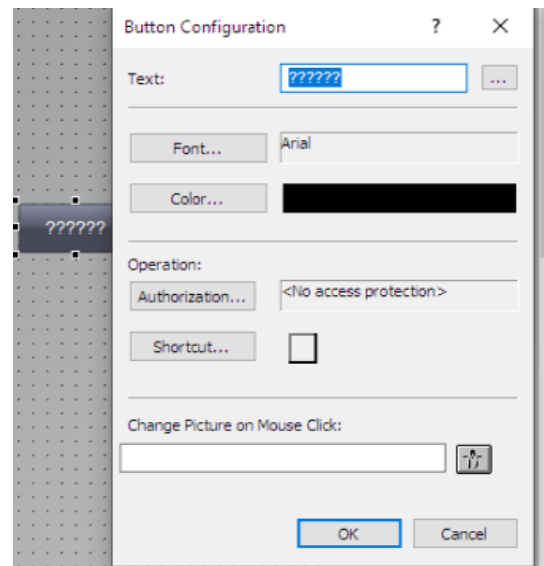
HMI Navigation Design via Direct Connection



To allow operators to seamlessly move between different process screens, trends, and analytical grids in runtime, I implemented a dedicated navigation interface using **Direct Connection** mechanisms within the WinCC **Graphics Designer** editor.

I opened the target graphical screen inside the Graphics Designer workspace. From the "Windows Objects" palette layout, I selected the **Button** component and positioned it onto the drawing canvas.

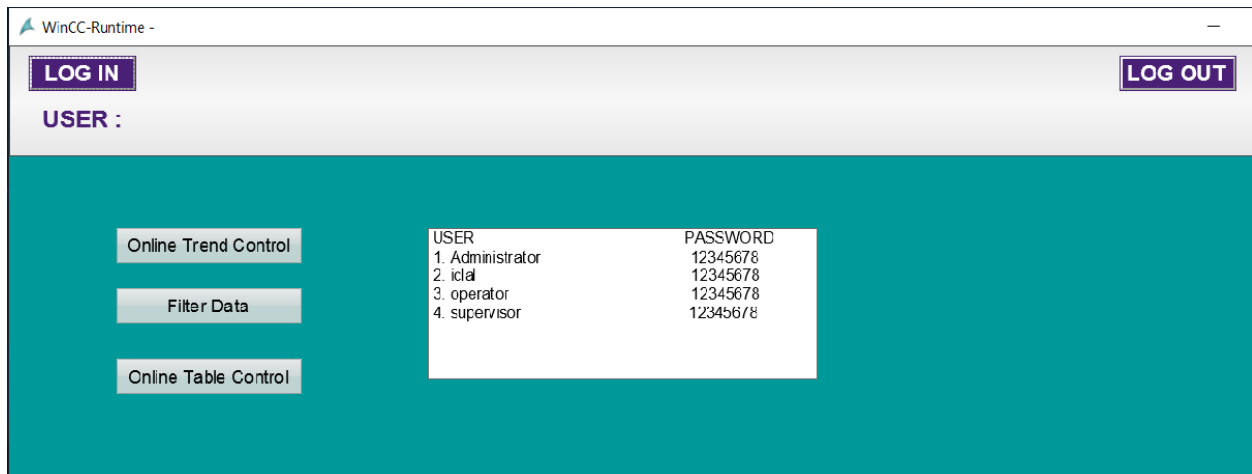
I double-clicked the button object to open its configuration dashboard. Inside the *Button Configuration* window, I defined the primary layout properties—such as modifying the visual typography, background colors, and assignable hotkey shortcuts. To link this object to a screen change operation, I accessed the event properties tab and initialized a **Direct Connection** command triggered on the *Mouse Click* event.



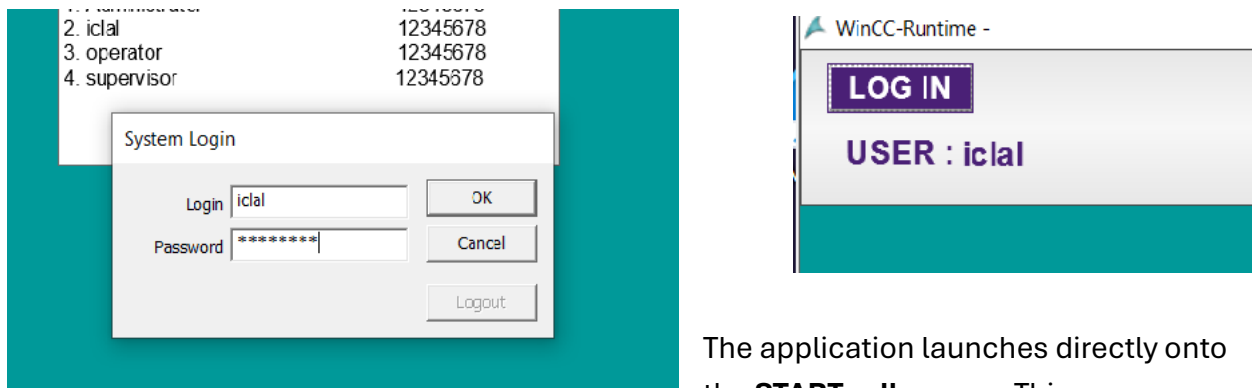
Upon assigning the mouse click operation, from the compiled local file system, I mapped the action directly to my primary SCADA screen, *icla1_graphic.pdl*. This ensures that when an operator clicks this specific button during runtime, WinCC instantly executes a page transition, safely redirecting the user to the main *icla1_graphic.pdl* process.

Runtime Security Validation and Dynamic User Login Testing

To verify that the role-based authorization matrix configured inside the User Administrator framework operates flawlessly during live execution, I conducted comprehensive testing within the WinCC Runtime environment.

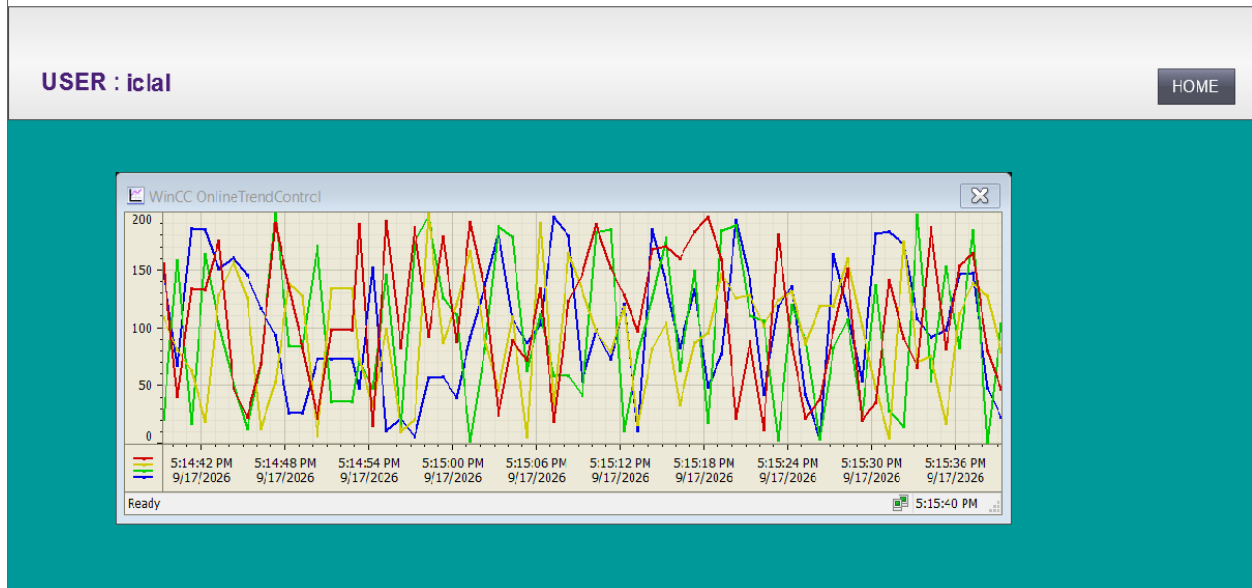


To organize the user interface logically, I structured the SCADA application into a multi-page hierarchy within WinCC Graphics Designer. This layout allows operators to switch smoothly between the welcome dashboard, security gates, historical trends, and advanced database filtering panels.



The application launches directly onto the **START.pdl** screen. This serves as the primary gateway and welcome interface. It features the master **LOG IN** and **LOG OUT** buttons, a dynamic text block tracking the active credentials, and navigation shortcuts like *Online Trend Control*, *Filter Data*, and *Online Table Control*. To help during the testing phase, a temporary cheat-sheet grid listing the user accounts (Administrator, iclal, operator, supervisor) and their passwords were placed in the center of the canvas.

Clicking the trend shortcuts routes the user directly to the **OnlineTrendControl.pdl** and **OnlineTableControl.pdl** screens. These screens host the active ActiveX chart components that plot the high-precision float variables.



Once a secure login is verified, a custom, purple-themed **HOME** button appears in the top-right corner of these pages. This button uses a Direct Connection handle to instantly flush the current view and redirect the operator back to the root START.pdl cockpit.

USER : iclal

HOME

WinCC OnlineTableControl

	Date - Time	Status	Pressure	Temperature	Total
6	9/17/2026 5:11:30	194.94613483077	52.51625:106296	18.878749961851	77.81609546:897
7	9/17/2026 5:11:31	142.44819483015	28.278450880459	90.688802758873	165.3986175:152
8	9/17/2026 5:11:32	120.16968291268	166.032898953211	18.372142704550	99.531896420178
9	9/17/2026 5:11:33	3.5584582049012	102.505569627975	98.068178350169	12.652974028748
10	9/17/2026 5:11:34	32.636494033631	70.80294:9843135	180.15076143681	90.597247230445
11	9/17/2026 5:11:35	4.9867244483779	118.7414:666921	165.18549150059	122.18390453810
12	9/17/2026 5:11:36	31.269264809105	84.304330576494	149.62004455702	162.52092959379
13	9/17/2026 5:11:37	180.56581316568	45.442060609759	41.480758079775	87.997070223090
14	9/17/2026 5:11:38	91.482284005249	62.605670339060	15.747550889614	21.051667836542
15	9/17/2026 5:11:39	148.57631153294	147.80724509415	129.85625782036	26.038392284920
16	9/17/2026 5:11:40	88.509781182287	10.559404278695	34.998626667073	133.28043458357
17	9/17/2026 5:11:41	149.90691854609	141.9904:718802	100.70497756889	158.39808648945
18	9/17/2026 5:11:42	96.8:0815759758	18.73226:116367	140.107425153355	99.94201483:995
19	9/17/2026 5:11:43	48.841822565385	138.86532181768	79.927976317636	113.1076998:994
20	9/17/2026 5:11:44	60.585345011749	112.03344828638	22.058778649250	194.22589800714

Ready Row 20 5:11:44 PM

LOG IN

USER : iclal

HOME

Start Date-Time

End Date-Time

September 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
31	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4
5	6	7	8	9	10	11

Today: 9/17/2026

September 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
31	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4
5	6	7	8	9	10	11

Today: 9/17/2026

tag_choice

2 :14:14 PM

2 :15:14 PM

FILTER FROM backup-db

export

Tag ID	Tag Name
☐ 1	Pressure
☐ 2	Status
☐ 3	Temperature
☐ 4	Tctal
☐ 11	deneme
☐ 13	new_test
☐ 14	Icbl_test
☐ 15	test_tag01
☐ 16	test_tag02
☐ 17	test_tag03
☐ 18	test_tag04

The most complex data layer is hosted on the **iclal_test.pdl** screen, which is accessed by clicking the *Filter Data* button. This panel isolates the historical data filtering controls, including the custom dual-calendar date pickers, time intervals, the standalone tag directory list, and the specialized **FILTER FROM backup-db** and **export** buttons. If a restricted user (like operator) attempts to navigate here from the home screen, the runtime engine blocks the transition and throws the authorization warning, keeping this file protected. Filter button's VBS script can be found directly within the provided **GitHub link**.

LOG IN

LOG OUT

USER : operator

Online Trend Control

Filter Data

Online Table Control

USER	PASSWORD
1. Administrator	12345678
2. iclal	12345678
3. operator	12345678
4. supervisor	12345678

Operator authorization

No operator authorization

OK